板

jimmyywang

2026 年 10 月 26 日

目录

1 数据结构 (Data Structures) 4
- 1.1 哈希表 . . . 4
- 1.2 fhq-treap . . . 5
- 1.3 RMQ (ST 表) . . . 7
- 1.4 笛卡尔树 (Cartesian Tree) . . . 7
- 1.5 树链剖分 (Heavy-Light Decomposition) . . . 8
- 1.6 LCT (Link-Cut Tree) . . . 9
- 1.7 李超线段树 . . . 11

2 图论 (Graph Theory) 13
- 2.1 连通性 (Connectivity) . . . 13
  - 2.1.1 Tarjan 求 SCC (Strongly Connected Components) . . . 13
  - 2.1.2 仅加边动态 scc . . . 13
  - 2.1.3 点双 / 边双 (Biconnected Components) . . . 15
  - 2.1.4 圆方树 (Round-Square Tree) . . . 16
- 2.2 最短路 (Shortest Path) . . . 17
  - 2.2.1 负环 (Negative Cycle Detection - SPFA) . . . 17
- 2.3 欧拉路径 (Eulerian Path) . . . 17
- 2.4 网络流 (Network Flow) . . . 18
  - 2.4.1 Dinic (Max Flow) . . . 18
  - 2.4.2 MCMF (Min Cost Max Flow) . . . 19
  - 2.4.3 网络流经典模型 . . . 20
- 2.5 图匹配 (Graph Matching) . . . 21
  - 2.5.1 二分图最小权完美匹配 (KM / SSP) . . . 21
- 2.6 树上算法 (Tree Algorithms) . . . 23
  - 2.6.1 树的重心 (Tree Centroid) . . . 23
  - 2.6.2 树上离某个点最远的点 . . . 23
  - 2.6.3 斯坦纳树 (Steiner Tree) . . . 24
- 2.7 2-SAT . . . 24

3 动态规划 (Dynamic Programming) 25
3.1 WQS 二分 . . . 25
3.2 决策单调性与二分队列 . . . 26

4 字符串 (String Algorithms) 27
4.1 字符串哈希 (String Hashing) . . . 27
4.2 KMP . . . 28
4.3 Manacher . . . 28
4.4 AC 自动机 (Aho-Corasick) . . . 28
4.5 后缀数组 (Suffix Array - SA) . . . 29
4.6 回文自动机 (Palindrome Automaton - PAM) . . . 30

5 数学 (Mathematics) 32
5.1 数论 (Number Theory) . . . 32
5.1.1 Exgcd (Extended Euclidean Algorithm) . . . 32
5.1.2 万能欧几里得 . . . 32
5.1.3 ExCRT (Extended Chinese Remainder Theorem) . . . 33
5.1.4 扩展欧拉定理 (Euler's Totient Theorem) . . . 33
5.1.5 Pollard-Rho . . . 34
5.1.6 Dirichlet 前缀和 (Dirichlet Prefix Sum) . . . 35
5.1.7 杜教筛 (Du's Sieve) . . . 35
5.1.8 Min25 . . . 36
5.1.9 单位根反演 . . . 37
5.2 组合数学 (Combinatorics) . . . 37
5.2.1 组合数 (Binomial Coefficients) . . . 37
5.2.2 Lucas 定理 (Lucas's Theorem) . . . 38
5.2.3 ExLucas (Extended Lucas Theorem) . . . 38
5.2.4 min-max 容斥 . . . 39
5.2.5 反演 . . . 39
5.3 线性代数 (Linear Algebra) . . . 40
5.3.1 高斯消元 (Gaussian Elimination) . . . 40
5.3.2 行列式 . . . 41
5.3.3 矩阵树定理 . . . 41
5.3.4 BEST 定理 (BEST Theorem) . . . 42
5.3.5 LGV 引理 (Weighted LGV Lemma) . . . 43
5.4 多项式 (Polynomials) . . . 43
5.4.1 Bostan-Mori . . . 45
5.5 数值方法 (Numerical Methods) . . . 45
5.5.1 自适应辛普森 (Adaptive Simpson's Rule) . . . 45

6 Misc 46
6.1 C++ 手动上下取整 . . . 46
6.2 单点修改区间摩尔投票 . . . 46
6.3 双栈模拟 dequeue . . . 47

6.4 测速 47
6.5 KKT 47

1 数据结构 (Data Structures)

1.1 哈希表

线性探测哈希表，使用进程级随机种子的 splitmix64；平均常数复杂度，性能取决于容量和负载。原稿的特定机器测速不能视为通用结论。

```
using u64=unsigned long long;
u64 xorshift(u64 x) {
  static const u64 seed=chrono::steady_clock::now().time_since_epoch().count();
    x += seed+0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
}
template<typename K, typename V, int M=(1<<20)>
struct HashTable{
  static_assert(M>0 && (M&(M-1))==0);
    static const int MASK=M-1;
  int cnt=0;vector<int>e;
  bitset<M> used;
  K k[M]; V v[M];
  void clear(){
    for(auto x:e)used[x]=0;
    e.clear();cnt=0;
  }
  int size(){return cnt;}
  int idx(K x){
    int u=xorshift(x)&MASK;
    for(int step=0;step<M;step++,u=(u+1)&MASK)
            if(!used[u]||k[u]==x)return u;
        return -1; // full table: never probe forever
  }
  V* find(K x){
    int i=idx(x);
    if(i<0||!used[i])return nullptr;
    return &v[i];
  }
  V& operator [](K x){
    int i=idx(x);
        if(i<0)throw overflow_error("HashTable capacity exhausted");
    if(!used[i]){
      used[i]=1;cnt++;
      e.pb(i);
      k[i]=x,v[i]=V{};
    }return v[i];
  }
};
```

使用 operator[] 插入或访问，find 返回值指针。此处默认 K、V 为整数；其他键类型需要替换哈希函数，且不提供有序 map 操作。

使用条件与多测 每组开始调用 `clear()`；新对象的 bitset 自带零初始化。线性探测仅支持插入、查找和整体清空，不支持直接删槽；建议负载率低于 0.7，高负载会退化，最坏单次 $O(M)$。该结构无序，不能代替 map 的有序操作。大对象应放在静态区或堆上。

1.2 fhq-treap

1. 值域分裂

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
struct node{
    ll ls,rs;
    ll val; unsigned long long rnd;
    ll sz,num;
}t[1000010];ll cur=0;
ll newnode(ll val){
    assert(cur+1<(ll)size(t));
    t[++cur]={0,0,val,rng(),1,1};
    return cur;
}void upd(ll p){
    t[p].sz=t[t[p].ls].sz+t[t[p].rs].sz+t[p].num;
}void split(ll p,ll k,ll &x,ll &y){
    if(!p){x=y=0;return;}
    if(t[p].val<=k)x=p,split(t[p].rs,k,t[x].rs,y);
    else y=p,split(t[p].ls,k,x,t[y].ls);
    upd(p);
}ll merge(ll x,ll y){
    if(!x||!y)return x^y;
    if(t[x].rnd<t[y].rnd){
        t[x].rs=merge(t[x].rs,y);
        upd(x);return x;
    }else{
        t[y].ls=merge(x,t[y].ls);
        upd(y);return y;
    }
}
ll rt;
void init(){cur=rt=0;t[0]={};}
void split_lt(ll p,ll k,ll &x,ll &y){
    if(!p){x=y=0;return;}
    if(t[p].val<k)x=p,split_lt(t[p].rs,k,t[x].rs,y);
    else y=p,split_lt(t[p].ls,k,x,t[y].ls);
    upd(p);
}
void insert(ll val){
    ll x,y,z;
    split_lt(rt,val,x,y);split(y,val,y,z);
    if(y==0)y=newnode(val);else t[y].num++,t[y].sz++;
    rt=merge(merge(x,y),z);
}void del(ll val){
    ll x,y,z;
    split_lt(rt,val,x,y);split(y,val,y,z);
    if(y==0){rt=merge(x,z);return;}
    t[y].num--,t[y].sz--;
    if(t[y].num==0)rt=merge(x,z);
    else rt=merge(merge(x,y),z);
}
ll rk(ll val){// 1 + (# of < val)
    ll x,y;split_lt(rt,val,x,y);
    ll res=t[x].sz+1;rt=merge(x,y);
    return res;
}
ll findkth(ll p,ll k){
    if(k<1||k>t[p].sz)throw out_of_range("kth");
    while(1){
```

```
        if(t[t[p].ls].sz>=k)p=t[p].ls;
        else if(t[t[p].ls].sz+t[p].num>=k)return t[p].val;
        else k-=t[t[p].ls].sz+t[p].num,p=t[p].rs;
    }
}
ll pred(ll val){ // max_{x<val} x
    ll x,y;split_lt(rt,val,x,y);
    if(!x){rt=merge(x,y);throw out_of_range("no predecessor");}
    ll res=findkth(x,t[x].sz);
    rt=merge(x,y);
    return res;
}ll succ(ll val){ // min_{x>val} x
    ll x,y;split(rt,val,x,y);
    if(!y){rt=merge(x,y);throw out_of_range("no successor");}
    ll res=findkth(y,1);
    rt=merge(x,y);
    return res;
}
```

2. 排名分裂

```
ll n,m,cnt;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
struct node{
    int ls,rs;
    ll val; unsigned long long rnd;
    int sz,num;
    int tg;// 打了 tag，可以不用。这里是区间 reverse 板子。
}t[100010];
#define ls(p) (t[(p)].ls)
#define rs(p) (t[(p)].rs)
int newnode(ll x){
    assert(cnt+1<(ll)size(t));
    t[++cnt]={0,0,x,rng(),1,1,0};
    return cnt;
}
void upd(int p){t[p].sz=t[ls(p)].sz+t[rs(p)].sz+t[p].num;}
void pd(int p){
    if(t[p].tg==0)return;
    swap(ls(p),rs(p));
    if(ls(p))t[ls(p)].tg^=1;
    if(rs(p))t[rs(p)].tg^=1;
    t[p].tg=0;
}
void split(int p,int sz,int &x,int &y){
    if(!p){x=y=0;return;}pd(p);
    if(t[ls(p)].sz<sz)x=p,split(rs(p),sz-t[ls(p)].sz-t[p].num,rs(p),y);
    else y=p,split(ls(p),sz,x,ls(p));
    upd(p);
}int merge(int x,int y){
    if(!x||!y)return x+y;pd(x),pd(y);
    if(t[x].rnd<t[y].rnd){rs(x)=merge(rs(x),y);upd(x);return x;}
    else {ls(y)=merge(x,ls(y));upd(y);return y;}
}
void out(int p){
    if(!p)return;pd(p);
    out(ls(p)),cout<<t[p].val<<" ",out(rs(p));
}
ll rt;
void init(){cnt=rt=0;t[0]={};}
```

使用条件与多测 两个代码块为独立备选实现;每组开始调用各自的 `init()`,复用节点时由 newnode 整体赋值，不必清空整个池。节点池容量按单组累计新建数计算，删除不会回收编号。排名分裂要求每个节点 num 为 1、分裂数量合法；查询排名越界或不存在前驱 / 后继会抛异常，调用方应事先判断。期望 $O(\log n)$，随机退化及递归深度仍需留意。

1.3 RMQ (ST 表)

ST 表 (Sparse Table)，用于 O(1) 查询区间最值 (RMQ)。

```
ll s[100010][20];
ll lg[100010];
ll get(ll l,ll r){
  ll L=r-l+1,Lg=lg[L];
  return max(s[l][Lg],s[r-(1<<Lg)+1][Lg]);
}
int main(){
  n=d,m=d;lg[1]=0;assert(1<=n&&n<=100000);
  f(i,1,n)s[i][0]=d;
  f(j,1,19)f(i,1,n){
    if(i+(1<<(j-1))>n)s[i][j]=s[i][j-1];
    else s[i][j]=max(s[i][j-1],s[i+(1<<(j-1))][j-1]);
  }f(i,2,n)lg[i]=lg[i/2]+1;
}
```

使用条件与多测 建表完整覆盖本次有效区间，令 lg[1]=0 后重算到 n 即可多测复用，无须整表清零。要求 $1 \le l \le r \le n \le 100000$；预处理 $O(n \log n)$，查询 $O(1)$。重叠区间合并要求幂等性，不能直接替换为求和。

1.4 笛卡尔树 (Cartesian Tree)

笛卡尔树 (Cartesian Tree)，一种基于键值和权值的二叉树，使用栈 O(n) 线性构建。

```
ll a[10000010];
int l[10000010],r[10000010];
long long x,y;
int st[10000010],tp;
int main(){
  n=d;tp=0;
    f(i,1,n)l[i]=r[i]=0;
    if(n==0)return 0;
    f(i,1,n)a[i]=d;
  st[++tp]=1;
  f(i,2,n){
    while(tp&&a[i]<a[st[tp]])l[i]=st[tp--];
    if(tp)r[st[tp]]=i;st[++tp]=i;
  }
}
```

使用条件与多测 每组必须清空当前 $[1, n]$ 的左右儿子与栈顶（代码已补）；根为构建后的 st[1]，同值按下标先后稳定处理。已将栈和左右儿子改为 int，内存从约 320 MB 降至约 200 MB；容量仍应按题目缩减；值的类型按题目决定。

1.5 树链剖分 (Heavy-Light Decomposition)

树链剖分，将树上问题转化为序列问题，常配合线段树使用。

```
ll dfn[100010],de[100010],fa[100010],son[100010];
ll sz[100010],top[100010],rk[100010];
void init_hld(int nodes){
    n=nodes;tim=cnt=0;sz[0]=de[0]=0;
    fill(hd,hd+n+1,0);
}
void dfs1(ll u,ll fat){
    fa[u]=fat;de[u]=de[fat]+1;sz[u]=1;son[u]=0;
    for(int i=hd[u];i;i=e[i].nx){
        ll v=e[i].v;if(v==fat)continue;
        dfs1(v,u);
        if(sz[v]>sz[son[u]])son[u]=v;
        sz[u]+=sz[v];
    }
}void dfs2(ll u,ll tp){
    top[u]=tp;dfn[u]=++tim;rk[tim]=u;
    if(son[u])dfs2(son[u],tp);
    for(int i=hd[u];i;i=e[i].nx){
        ll v=e[i].v;if(v==fa[u]||v==son[u])continue;
        dfs2(v,v);
    }
}
ll a[100010];
#define ls(p) (p<<1)
#define rs(p) (p<<1|1)
ll t[100010<<2];
ll tg[100010<<2];
void upd(ll p){t[p]=(t[ls(p)]+t[rs(p)])%mod;}
void build(ll l,ll r,ll p){
    tg[p]=0;
    if(l==r){t[p]=(a[rk[l]]%mod+mod)%mod;return;}
    ll mid=l+r>>1;
    build(l,mid,ls(p));
    build(mid+1,r,rs(p));
    upd(p);
}
void pd(ll p,ll l,ll r){
    if(tg[p]==0)return;ll mid=l+r>>1;
    tg[ls(p)]=(tg[ls(p)]+tg[p])%mod;
    tg[rs(p)]=(tg[rs(p)]+tg[p])%mod;
    t[ls(p)]=(t[ls(p)]+tg[p]*(mid-l+1))%mod;
    t[rs(p)]=(t[rs(p)]+tg[p]*(r-mid))%mod;
    tg[p]=0;
}
void add(ll l,ll r,ll p,ll L,ll R,ll k){
    if(L<=l&&r<=R){t[p]=(t[p]+k*(r-l+1))%mod;tg[p]=(tg[p]+k)%mod;return;}
    pd(p,l,r);
    ll mid=l+r>>1;
    if(mid>=L)add(l,mid,ls(p),L,R,k);
    if(mid<R)add(mid+1,r,rs(p),L,R,k);
    upd(p);
}ll get(ll l,ll r,ll p,ll L,ll R){
    if(L<=l&&r<=R)return t[p];
    pd(p,l,r);
    ll mid=l+r>>1,res=0;
    if(mid>=L)res+=get(l,mid,ls(p),L,R);
```

```
    if(mid<R)res+=get(mid+1,r,rs(p),L,R);
    return res%mod;
}
void chad(ll u,ll v,ll k){
    k=(k%mod+mod)%mod;
    while(top[u]!=top[v]){
        if(de[top[u]]<de[top[v]])swap(u,v);
        add(1,n,1,dfn[top[u]],dfn[u],k);
        u=fa[top[u]];
    }if(dfn[u]>dfn[v])swap(u,v);
    add(1,n,1,dfn[u],dfn[v],k);
}ll chs(ll u,ll v){ll res=0;
    while(top[u]!=top[v]){
        if(de[top[u]]<de[top[v]])swap(u,v);
        res=(res+get(1,n,1,dfn[top[u]],dfn[u]))%mod;
        u=fa[top[u]];
    }if(dfn[u]>dfn[v])swap(u,v);
    res+=get(1,n,1,dfn[u],dfn[v]);
    return res%mod;
}
```

使用条件与多测 每组读边前调用 init_hld(n)，其中 cnt 为前向星边计数；读完后依次 dfs1(root,0)、dfs2(root,root)、build(1,n,1)。build 已逐节点清零懒标记，dfs1 已清 son。仅用于一棵树，路径操作要求两点连通；递归在链上可能栈溢出，深链须改为显式栈。mod 必须为正，乘法及两个余数相加须在 ll 范围内，否则改用 `__int128`。

1.6 LCT (Link-Cut Tree)

动态森林，维护路径异或；操作均摊 $O(\log n)$。

这里求得是链 xor

```
#include<bits/stdc++.h>
using namespace std;
#define ll int
#define f(i,a,b) for(ll i=a;i<=b;i++)
#define wt int tt=d;while(tt--)
#define py puts("Yes")
#define pn puts("No")
#define pritnf printf
#define fe(i,e) for(int i=0;i<e.size();i++)
#define vi vector<ll>
inline ll rd() {
  ll x=0,f=1;
  char c=getchar();
  while(!isdigit(c)){if(c=='-')f=-1;c=getchar();}
  while(isdigit(c))x=x*10+c-'0',c=getchar();
  return x*f;
}
ll dx[4]={0,1,0,-1};
ll dy[4]={1,0,-1,0};
#define d rd()
#define pb push_back
const ll N=300010;
struct edge{ll v,w,nx;}e[N<<1];
ll hd[N],cnt;
void add(ll u,ll v,ll w){e[++cnt]=(edge){v,w,hd[u]};hd[u]=cnt;}
ll qp(ll a,ll b,ll p){
```

```
  ll ans=1;while(b){
    if(b&1)ans=ans*a%p;
    a=a*a%p;b>>=1;
  }return ans;
}ll n,m;
ll Rt,tot;
ll fa[100010],ch[100010][2];
ll val[100010],sum[100010];
ll tg[100010];
#define ls(p) ch[p][0]
#define rs(p) ch[p][1]
bool isr(ll p){return (p!=ls(fa[p])&&p!=rs(fa[p]));}
void upd(ll p){sum[p]=sum[ls(p)]^sum[rs(p)]^val[p];}
bool get(ll p){return p==rs(fa[p]);}
void upt(ll p){swap(ls(p),rs(p));tg[p]^=1;}
void pd(ll p){
  if(!tg[p])return;
  if(ls(p))upt(ls(p));
  if(rs(p))upt(rs(p));
  tg[p]=0;
}void rot(ll x){
  ll y=fa[x],z=fa[y],t=get(x);
  if(!isr(y))ch[z][y==rs(z)]=x;
  ch[y][t]=ch[x][t^1];if(ch[x][t^1])fa[ch[x][t^1]]=y;
  ch[x][t^1]=y;fa[y]=x,fa[x]=z;upd(y);upd(x);
}void pda(ll x){
    static vector<int> st;st.clear();
    for(int y=x;;y=fa[y]){st.push_back(y);if(isr(y))break;}
    while(!st.empty()){pd(st.back());st.pop_back();}
}
void splay(ll x){//cout<<x<<endl;
  pda(x);//cout<<x<<endl;
  while(!isr(x)){
    ll y=fa[x],z=fa[y];
    if(!isr(y))rot(get(x)==get(y)?y:x);
    rot(x);
  }upd(x);
}void access(ll x){
  for(int y=0;x;y=x,x=fa[x]){
    splay(x);
    // cout<<x<<" "<<y<<endl;
    rs(x)=y,upd(x);
  }
}void mkr(ll x){
  access(x);
  splay(x);upt(x);
}ll find(ll x){
  access(x),splay(x);
  for(pd(x);ls(x);pd(x))x=ls(x);
  splay(x);return x;
}void split(ll x,ll y){
  mkr(x);access(y);splay(y);
}ll link(ll x,ll y){
  // cout<<x<<" "<<y<<endl;
  mkr(x);
  if(find(y)==x)return 0;
  fa[x]=y;splay(y);return 1;
}ll cut(ll x,ll y){
  split(x,y);
```

```
    if(ls(y)!=x||rs(x))return 0;
    ls(y)=fa[x]=0;upd(y);return 1;
}
int main(){
  n=d,m=d;
    f(i,0,n){fa[i]=ch[i][0]=ch[i][1]=tg[i]=val[i]=sum[i]=0;}
    f(i,1,n)sum[i]=val[i]=d;
  while(m--){
    ll o=d,x=d,y=d;
    if(o==0)split(x,y),printf("%d\n",sum[y]);
    if(o==1)link(x,y);if(o==2)cut(x,y);
    if(o==3)access(x),splay(x),val[x]=y,upd(x);
  }
  return 0;
}
```

使用条件与多测 每组先执行 main 中的节点初始化，再赋点权；零号节点始终为全零哨兵。main 的单组逻辑可提取为 solve 后放入多测循环。路径查询前须确认 find(x)==find(y)；link 拒绝成环，cut 只删除真实存在的边。点权使用 int；本模板是异或而非求和。

1.7 李超线段树

整数横坐标、整数系数的动态李超线段树，求最小值。使用 128 位精确比较，避免浮点误差及空节点的有限 inf 干扰。整条直线插入 $O(\log C)$，线段插入 $O(\log^2 C)$，单点查询 $O(\log C)$。

```
struct LiChao {
    using i128=__int128_t;
    static constexpr int LO=-1000000000,HI=1000000000;
    static constexpr i128 INF=(i128(1)<<120);
    struct Line{ll k,b;};
    struct Node{int l=0,r=0,id=0;};
    vector<Line> lines;
    vector<Node> tr;
    int root=0;
    LiChao(){clear();}
    void clear(){lines.assign(1,{0,0});tr.assign(1,{});root=0;}
    i128 value(int id,int x)const{
        return id?i128(lines[id].k)*x+lines[id].b:INF;
    }
    int node(){tr.push_back({});return int(tr.size())-1;}
    int insert(int p,int l,int r,int id){
        if(!p)p=node();
        int mid=l+int((ll(r)-l)/2);
        if(value(id,mid)<value(tr[p].id,mid))swap(id,tr[p].id);
        if(l==r||!id)return p;
        if(value(id,l)<value(tr[p].id,l)){
            int child=insert(tr[p].l,l,mid,id);tr[p].l=child;
        }else if(value(id,r)<value(tr[p].id,r)){
            int child=insert(tr[p].r,mid+1,r,id);tr[p].r=child;
        }
        return p;
    }
    int cover(int p,int l,int r,int L,int R,int id){
        if(L<=l&&r<=R)return insert(p,l,r,id);
        if(!p)p=node();
        int mid=l+int((ll(r)-l)/2);
        if(L<=mid){int c=cover(tr[p].l,l,mid,L,R,id);tr[p].l=c;}
```

```
        if(R>mid){int c=cover(tr[p].r,mid+1,r,L,R,id);tr[p].r=c;}
        return p;
    }
    void add_segment(int L,int R,ll k,ll b){ // inclusive [L,R]
        L=max(L,LO);R=min(R,HI);if(L>R)return;
        lines.push_back({k,b});
        root=cover(root,LO,HI,L,R,int(lines.size())-1);
    }
    i128 query(int x)const{
        assert(LO<=x&&x<=HI);
        int p=root,l=LO,r=HI;i128 ans=INF;
        while(p){
            ans=min(ans,value(tr[p].id,x));if(l==r)break;
            int mid=l+int((ll(r)-l)/2);
            if(x<=mid)p=tr[p].l,r=mid;
            else p=tr[p].r,l=mid+1;
        }
        return ans;
    }
};
```

使用条件与多测 每组调用 clear()（构造时自动调用）；节点池按实际使用增长，无旧节点残留。若题目给 $[l,r)$，先判断 $l<r$ 再插入 $[l,r-1]$。query 返回 INF 表示无覆盖，否则返回 128 位整数，须按值域选择输出方法，不要未经检查强转 ll。此版本不支持实数横坐标；实数题需另写浮点版本及误差策略。

2 图论 (Graph Theory)

2.1 连通性 (Connectivity)

2.1.1 Tarjan 求 SCC (Strongly Connected Components)

Tarjan 算法，用于在线性时间内求解有向图的强连通分量。

```
ll dfn[100010],low[100010],cur;
ll bel[100010],scc;
ll st[100010],tp;
bool ins[100010];
vi e[100010];
void dfs(ll u){
  dfn[u]=low[u]=++cur;
  st[++tp]=u,ins[u]=1;
  fe(i,e[u]){
    ll v=e[u][i];
    if(!dfn[v])dfs(v),low[u]=min(low[u],low[v]);
    else if(ins[v])low[u]=min(low[u],dfn[v]);
  }if(low[u]==dfn[u]){
    scc++;ll now;while(1){
      now=st[tp--];
      ins[now]=0,bel[now]=scc;
      if(now==u)break;
    }
  }
}
void init(int n){
    cur=scc=tp=0;
    for(int i=0;i<=n;i++){
        dfn[i]=low[i]=bel[i]=0;ins[i]=false;e[i].clear();
    }
}
```

使用条件与多测 每组读边前调用下面的 init(n)，读完后遍历所有顶点：未访问则 dfs(i)。编号从 1 开始，bel 按逆拓扑序编号；一条 $u\to v$ 的跨分量边满足 bel[u]>bel[v]。时间 $O(n+m)$。深链递归可能栈溢出，按评测栈限制改显式栈。

2.1.2 仅加边动态 scc

每条边的生效时间是不小于他的加入时间的，整体二分这个时刻，维护 (l,r,E) 表示生效时间在 $[l,r]$ 内的边。加入 $[l,mid]$ 中的边考察 scc 变化情况判断生效时间在左区间还是右区间（也可能不生效）。

```
int dfn[100010],low[100010],tim;
int st[100010],tp;
bool in[100010];
int bel[100010],scc;
```

```
vector<int>e[100010];
vector<int>node;
void dfs(int u){
    dfn[u]=low[u]=++tim;
    st[++tp]=u,in[u]=1;
    for(auto v:e[u]){
        if(!dfn[v])dfs(v),low[u]=min(low[u],low[v]);
        else if(in[v])low[u]=min(low[u],dfn[v]);
    }if(dfn[u]==low[u]){
        scc++;int now;while(1){
            now=st[tp--];
            bel[now]=scc;in[now]=0;
            if(now==u)break;
        }
    }
}
void clear(){
    tim=0,tp=0;scc=0;
    for(auto u:node)dfn[u]=low[u]=0,bel[u]=0,in[u]=false,e[u].clear();
    node.clear();
}
int U[100010],V[100010];
vector<int>con[100010];
int fa[100010],dsz[100010];
void init_dsu(int n){for(int i=1;i<=n;i++)fa[i]=i,dsz[i]=1;}
int find(int x){
    if(fa[x]==x)return x;
    return fa[x]=find(fa[x]);
}
void un(int u,int v){
    if(find(u)==find(v))return;
    u=find(u),v=find(v);
    if(dsz[u]>dsz[v])swap(u,v);fa[u]=v;dsz[v]+=dsz[u];
}
void work(int l,int r,vector<array<int,3>>&E){// E 中储存 (u,v,i)，表示在加入全部 $[1,l-1]$ 的边后，
    u 和 v 所在的 scc 编号。这样接着在这个图上做可以简化。
    // printf("Working on [%d, %d]\n",l,r);
    // for(auto [u,v,i]:E)printf("(%d, %d, %d) ",u,v,i);cout<<endl;
    if(l>r||E.empty())return;
    int mid=(l+r)>>1;
    vector<array<int,3>> e1,e2;e1.clear(),e2.clear();
    for(auto [u,v,i]:E){
        node.pb(u),node.pb(v);
        if(i<=mid)e[u].pb(v);
    }
    for(auto u:node)if(!dfn[u])dfs(u);
    if(l==r){
        for(auto [u,v,i]:E){
            if(bel[u]==bel[v])un(U[i],V[i]);//有了“生效”状态后即可转化为并查集维护。
        }clear();
        return;
    }
    for(auto [u,v,i]:E){
        if(bel[u]!=bel[v])e2.pb({bel[u],bel[v],i});
        else if(i<=mid&&bel[u]==bel[v])e1.pb({u,v,i});
    }clear();
    work(l,mid,e1),work(mid+1,r,e2);
}
void init_dynamic_scc(int n){
```

```
    clear();init_dsu(n);tim=tp=scc=0;
    for(int u=0;u<=n;u++){
        dfn[u]=low[u]=bel[u]=0;in[u]=false;e[u].clear();con[u].clear();
    }
}
void solve_dynamic_scc(int n,const vector<pair<int,int>>& edges){
    assert(n<100010&&edges.size()<100010);init_dynamic_scc(n);
    vector<array<int,3>> E;
    for(int i=0;i<int(edges.size());i++){
        auto [u,v]=edges[i];U[i+1]=u;V[i+1]=v;E.push_back({u,v,i+1});
    }
    if(!E.empty())work(1,edges.size(),E);
}
```

使用条件与多测 这是离线计算前缀强连通合并事件的片段，不支持在线删边。每组先初始化并查集，再清 node 并清零 tim、tp、scc、dfn、low、in、bel 及邻接表；给 U[i],V[i] 赋原始端点，再建立 E=(U[i],V[i],i)，调用 work(1,m,E)，$m = 0$ 时跳过。con 若另有业务用途也须逐组清空。叶子中 un 的调用顺序按时间递增，可在该处记录事件；函数返回后仅保留最终并查集，不会自动输出各时刻答案。已用按大小合并避免并查集深链；Tarjan 递归深度仍按题目处理。总体按每层边数计 $O((n+m)\log m)$ 次图工作，空子问题已剪枝。

使用条件与多测 已补 init_dynamic_scc 与 solve_dynamic_scc 入口；仅求最终合并结果可直接调用后者。需要各时刻答案时，在 work 的叶子中记录事件，不能将算法返回后的最终 fa 当作历史状态。

2.1.3 点双 / 边双 (Biconnected Components)

同时求桥、边双、点双及点双边集。顶点从 1 编号，原始无向边从 0 编号，以边编号跳过父边，正确支持重边。自环不影响桥 / 点分割，点双边集约定忽略自环；若题目要求给自环分组，需另行定义。每组 init(n) 后加边、run()；run 可重复运行，时间空间 $O(n+m)$。

```
struct Biconnected {
    int n,timer=0;
    vector<pair<int,int>> edges;
    vector<vector<pair<int,int>>> g;
    vector<int> dfn,low,stack,component,seen;
    vector<char> bridge;
    vector<vector<int>> blocks,block_edges;
    void init(int size){n=size;edges.clear();g.assign(n+1,{});}
    int add(int u,int v){
        int id=edges.size();edges.push_back({u,v});
        if(u!=v){g[u].push_back({v,id});g[v].push_back({u,id});}
        return id;
    }
    void dfs(int u,int parent_edge){
        dfn[u]=low[u]=++timer;
        for(auto [v,id]:g[u]){
            if(id==parent_edge)continue;
            if(!dfn[v]){
                stack.push_back(id);dfs(v,id);low[u]=min(low[u],low[v]);
                if(low[v]>dfn[u])bridge[id]=true;
                if(low[v]>=dfn[u]){
                    blocks.push_back({});block_edges.push_back({});
                    int stamp=blocks.size();
```

```
                    for(;;){
                        int e=stack.back();stack.pop_back();
                        block_edges.back().push_back(e);
                        auto [a,b]=edges[e];
                        for(int x:{a,b})if(seen[x]!=stamp){
                            seen[x]=stamp;blocks.back().push_back(x);
                        }
                        if(e==id)break;
                    }
                }
            }else if(dfn[v]<dfn[u]){
                stack.push_back(id);low[u]=min(low[u],dfn[v]);
            }
        }
    }
    void run(){
        timer=0;dfn.assign(n+1,0);low.assign(n+1,0);seen.assign(n+1,0);
        bridge.assign(edges.size(),false);stack.clear();blocks.clear();block_edges.clear();
        for(int u=1;u<=n;u++)if(!dfn[u]){
            if(g[u].empty())blocks.push_back({u}),block_edges.push_back({});
            dfs(u,-1);
        }
        component.assign(n+1,0);int count=0;
        for(int u=1;u<=n;u++)if(!component[u]){
            vector<int> q{u};component[u]=++count;
            while(!q.empty()){
                int x=q.back();q.pop_back();
                for(auto [v,id]:g[x])if(!bridge[id]&&!component[v]){
                    component[v]=count;q.push_back(v);
                }
            }
        }
    }
};
```

使用条件与多测 blocks[k] 与 block_edges[k] 一一对应；只遍历该点双的边集才能维持线性总复杂度，不要反复扫描割点原邻接表。dfs 为递归实现，最大深度 n；大规模深链须使用显式栈或足够栈空间。

2.1.4 圆方树 (Round-Square Tree)

依赖前文 Biconnected：先 run()，再调用下列函数。圆点编号 $1..n$，第 k 个点双对应方点 $n+k+1$。孤立点保留为单个圆点，不额外生成一元方点；非连通图得到圆方森林。多测返回全新的邻接表，时间空间 $O(n+m)$。

```
vector<vector<int>> block_cut_forest(const Biconnected& b){
    vector<vector<int>> tree(b.n+b.blocks.size()+1);
    for(int k=0;k<int(b.blocks.size());k++){
        if(b.blocks[k].size()==1)continue;
        int square=b.n+k+1;
        for(int u:b.blocks[k]){tree[u].push_back(square);tree[square].push_back(u);}
    }
    return tree; // skipped singleton squares are unused IDs
}
```

2.2 最短路 (Shortest Path)

2.2.1 负环 (Negative Cycle Detection - SPFA)

检测整张有向图是否存在负环，包括与 1 不连通的分量。所有点以距离 0 入队，相当于添加超级源；每次松弛记录路径边数而非入队次数，达到 n 即判负环。局部状态支持直接多测；最坏 $O(nm)$，SPFA 不保证随机数据下的性能。

```
bool negative_cycle(const vector<vector<pair<int,ll>>>& g){
    int n=int(g.size())-1; // vertices 1..n
    vector<__int128_t> dist(n+1,0);
    vector<int> length(n+1,0);
    vector<char> in(n+1,true);queue<int> q;
    for(int i=1;i<=n;i++)q.push(i);
    while(!q.empty()){
        int u=q.front();q.pop();in[u]=false;
        for(auto [v,w]:g[u])if(dist[v]>dist[u]+w){
            dist[v]=dist[u]+w;length[v]=length[u]+1;
            if(length[v]>=n)return true;
            if(!in[v])q.push(v),in[v]=true;
        }
    }
    return false;
}
```

2.3 欧拉路径 (Eulerian Path)

有向图 Hierholzer，顶点从 1 编号；返回空 vector 表示无解。检查度数与实际使用边数，可处理非连通图、重边、自环；无边且 $n > 0$ 返回顶点 1。显式栈避免 $O(m)$ 深递归。局部变量支持多测；排序后输出字典序最小路径，时间 $O(n + m \log m)$，不要求字典序时可去掉排序，降为 $O(n + m)$。

```
vector<int> euler_directed(int n,const vector<pair<int,int>>& edges){
    if(n<=0)return {};
    vector<vector<int>> g(n+1);vector<int> in(n+1),out(n+1),cur(n+1);
    for(auto [u,v]:edges){g[u].push_back(v);out[u]++;in[v]++;}
    int start=-1,finish=-1;
    for(int u=1;u<=n;u++){
        if(out[u]-in[u]==1){if(start!=-1)return {};start=u;}
        else if(in[u]-out[u]==1){if(finish!=-1)return {};finish=u;}
        else if(in[u]!=out[u])return {};
        sort(g[u].begin(),g[u].end());
    }
    if((start==-1)!=(finish==-1))return {};
    if(start==-1){start=1;for(int u=1;u<=n;u++)if(out[u]){start=u;break;}}
    vector<int> st{start},path;
    while(!st.empty()){
        int u=st.back();
        if(cur[u]<int(g[u].size()))st.push_back(g[u][cur[u]++]);
        else path.push_back(u),st.pop_back();
    }
    if(path.size()!=edges.size()+1)return {};
    reverse(path.begin(),path.end());return path;
}
```

使用条件与多测 无向图需给每条边一个共享编号，经过时标记该编号而不是两个方向各算一次；要求奇度点恰为 0 或 2，起点选非零度点 / 奇度点，同样验证最终使用所有边。

2.4 网络流 (Network Flow)

2.4.1 Dinic (Max Flow)

Dinic 算法，用于高效求解网络最大流。

```
struct node{ll u,v,w,nx;}e[1000010];
ll hd[100010],cnt=1;
void add(ll u,ll v,ll w){e[++cnt]={u,v,w,hd[u]};hd[u]=cnt;}
void ad(ll u,ll v,ll w){add(u,v,w),add(v,u,0);}

ll de[100010],cur[100010];
ll dis[100010];
const ll inf=LLONG_MAX/4;
queue<ll>q;
ll nodes,S,T;
void init_dinic(int count,int source,int sink){
    nodes=count;S=source;T=sink;cnt=1;fill(hd,hd+nodes+1,0);
}
bool bfs(){
    while(!q.empty())q.pop();
    f(i,1,nodes)cur[i]=hd[i],de[i]=inf;de[S]=0;q.push(S);
    while(!q.empty()){
        ll u=q.front();q.pop();
        for(int i=hd[u];i;i=e[i].nx){
            ll v=e[i].v;if(e[i].w==0)continue;
            if(de[v]==inf){
                de[v]=de[u]+1;q.push(v);
                if(v==T)return 1;
            }
        }
    }return 0;
}
ll dfs(ll u,ll fl){
    if(u==T)return fl;
    ll res=0,k=0;
    for(ll &i=cur[u];i&&fl;i=e[i].nx){if(e[i].w==0)continue;
        ll v=e[i].v;if(de[v]==de[u]+1){
            k=dfs(v,min(fl,e[i].w));
            e[i].w-=k,e[i^1].w+=k;
            fl-=k,res+=k;
            if(!fl)break; // preserve a partially usable current arc
        }
    }return res;
}
ll n,m;
ll dinic(){
    ll res=0;
    if(S==T)return 0;
    while(bfs()){ll pushed;while((pushed=dfs(S,inf)))res+=pushed;}
    return res;
}
int main(){
    n=d,m=d,S=d,T=d;init_dinic(n,S,T);
    f(i,1,m){
        ll u=d,v=d,w=d;
        ad(u,v,w);
    }cout<<dinic();
    return 0;
}
```

使用条件与多测 每组读边前清 hd[0..nodes]、cnt=1；容量残量由 ad 重新赋值。重复 dinic() 返回新增流量而非历史总流量。容量非负，源汇应不同，流量和必须可用 ll 表示；nodes 包括超级源汇，边池容量按两倍有向边数算。DFS 可能深至点数；通用图复杂度 $O(V^2E)$，不能将单位容量图的更强界套到任意网络。

2.4.2 MCMF (Min Cost Max Flow)

upd on 25.10.12 已严肃进行势能优化 (使用 SPFA 预处理 h[ ] 数组，后续使用 Dijkstra)。

```
struct node{ll u,v,w,c,nx;}e[1000010];
ll hd[100010],cnt=1;
void add(ll u,ll v,ll w,ll c){e[++cnt]={u,v,w,c,hd[u]};hd[u]=cnt;}
void ad(ll u,ll v,ll w,ll c){add(u,v,w,c),add(v,u,0,-c);}
ll n,m,S,T,nodes;
ll cst;
void init_mcmf(int count,int source,int sink){
    nodes=count;S=source;T=sink;cnt=1;cst=0;fill(hd,hd+nodes+1,0);
}
ll pre[100010];
ll dis[100010];
ll h[100010];
const ll inf=LLONG_MAX/4;
ll arg(){
    ll u,v=T,fl=inf;
    while(v!=S)u=pre[v],fl=min(fl,e[u].w),v=e[u].u;
    v=T;while(v!=S)u=pre[v],e[u].w-=fl,e[u^1].w+=fl,v=e[u].u;
    cst+=fl*(dis[T]-(h[S]-h[T]));return fl;
}
struct myp{ll w,x;};
bool operator < (myp a,myp b){return a.w>b.w;}
bool in[100010];
bool dij(){
    priority_queue<myp>q;
    f(i,1,nodes)dis[i]=inf,pre[i]=in[i]=0;
    dis[S]=0;q.push({0,S});
    while(!q.empty()){
        ll u=q.top().x;q.pop();
        if(in[u])continue;in[u]=1;
        for(int i=hd[u];i;i=e[i].nx){
            if(e[i].w==0)continue;ll v=e[i].v;
            if(dis[u]+(e[i].c+h[u]-h[v])<dis[v]){
                dis[v]=dis[u]+(e[i].c+h[u]-h[v]);pre[v]=i;
                q.push({dis[v],v});
            }
        }
    }
    return dis[T]!=inf;
}
array<ll,2> mcmf(){
    if(S==T)return {0,0};
    ll fl=0;cst=0;
    while(dij()){
        fl+=arg();
        f(i,1,nodes)if(dis[i]!=inf)h[i]+=dis[i];
        // f(i,1,nodes)cout<<h[i]<<" ";cout<<endl;
    }return {fl,cst};
}
void spfa(ll s){
```

```
    queue<ll>q;
    fill(h,h+nodes+1,inf);
    fill(in,in+nodes+1,false);
    h[s]=0;q.push(s);in[s]=1;
    while(!q.empty()){
        ll u=q.front();q.pop();in[u]=0;
        for(int i=hd[u];i;i=e[i].nx){
            if(e[i].w==0)continue;ll v=e[i].v;
            if(h[u]+e[i].c<h[v]){
                h[v]=h[u]+e[i].c;pre[v]=i;
                if(!in[v])in[v]=1,q.push(v);
            }
        }
    }
}
int main(){
    n=d,m=d,S=d,T=d;init_mcmf(n,S,T);
    f(i,1,m){
        ll u=d,v=d,w=d,c=d;
        ad(u,v,w,c);
    }spfa(S);
    auto res=mcmf();
    cout<<res[0]<<" "<<res[1]<<endl;
    return 0;
}
```

使用条件与多测　每组重建 hd、cnt=1、cst=0，读完全部边后 spfa(S) 再 mcmf()。仅更新有限距离点的势能，避免不可达点反复加 INF 溢出；固定残量网络增广不会使原来不可达的顶点突然可达。若另行增删边或换源，必须重算势能。初始源可达残量图不得含负费用环，否则 SPFA 可能不终止；任意负环网络须用后文负权边转化或消圈算法。容量、距离、势能、流量乘费用及总费用须满足 ll 范围，INF 必须大于所有合法最短路。复杂度为初始 SPFA 加 $O(AE\log V)$，A 是增广次数，非强多项式。

2.4.3　网络流经典模型

上下界无费用流

1. 无源汇上下界可行流：钦定容量下界流满并计算这种情况下每个点的净流入量 in_u，新建超级源点 S 和超级汇点 T 用以模拟净流入的影响。如果 $in_u > 0$ 则连边 (S, u, in_u)，否则连边 $(u, T, -in_u)$。原图中其他边容量改为上下界之差。跑新图最大流，若满流则有可行流，把每条边原来的下界加上即可；否则无解。

2. 有源汇上下界可行流：新建一条 $(T, S, +\infty)$ 的边即转化为无源汇上下界可行流。

3. 有源汇上下界最大流：跑完无源汇的可行流后，断掉 $(T, S, +\infty)$ 的边后，残量网络上源汇之间（注意是原来的源汇不是新建的）继续跑最大流，加上即可。

4. 有源汇上下界最小流：跑完无源汇的可行流后，断掉 $(T, S, +\infty)$ 的边后，残量网络上汇点到源点跑最大流，即求最大退流，减去即可。

上下界费用流

以最小费用为例。若要求费用最大，全体费用取相反数以后做即可。鉴于可能出现负环，对于负权费用流后文会提到处理方式。以下默认费用为正。

1. 无源汇上下界最小费用可行流：同无费用版本，把每一步都算上费用，最大流换成最小费用最大流。

   注：费用全正的情况下，不需要跑最小费用退流。如果能退，则存在权为负的增广路，没理由不在最小费用最大流的时候被处理过。

2. 有源汇上下界最小费用可行流：新建一条 $(T, S, +\infty, 0)$ 的边即转化为无源汇上下界最小费用可行流。

3. 有源汇上下界最小费用最大流：跑完无源汇的最小费用可行流后，断掉 $(T, S, +\infty, 0)$ 的边后，残量网络上源汇之间继续跑最小费用最大流，加上即可。

4. 有源汇上下界最小费用最小流：跑完无源汇的最小费用可行流后，断掉 $(T, S, +\infty, 0)$ 的边后，残量网络上汇点到源点跑最小费用最大流，减去即可（费用是加上）。

负权费用流

其实没那么神秘。对于每条负权边 (u, v, c, w)，我们把它变成两条边：一条是 (u, v, c, w) 强制满流，另一条是 $(v, u, c, -w)$ 的退流边。强制满流直接当作上下界限制即可，在跑新图的时候负权边的容量会变成 0，等于没有这条边。这样所有费用都是正的。

同余网络流

现编的名字，即每条边流量都有一个全局的 $\bmod k$ 限制。把每条边的流量写成 $kx + y$，那么每个点处就会有关于 x 的流量平衡，由于 y 的净流入可能是 k 的正数倍 Δk，所有流量平衡有一个偏差项 Δ，与上下界一样用额外源汇模拟偏差即可在 x 上做流子。

最大权闭合子图

建立源汇 S, T，对正权点连边 (S, u, w_u)，负权点连边 $(u, T, -w_u)$，原图边 (u, v) 建 (u, v, ∞)，全图正权和减去最小割即为答案。

答案构造：在最大流结束的残量网络中，从源点沿正残量边可达的原图顶点组成所选闭合子图。

最长反链

对原图每个点拆点 (in_u, out_u)，原图边 (u, v) 改为 (out_u, in_v)，所有 in_u 点权为 -1，所有 out_u 点权为 1。此图最大权闭合子图即为最长反链。

这个图比较好理解，因为一个最大的闭合子图一定是由若干个 out_u 作为起点发出的（否则权值和非正），这几个 u 构成了一个反链。

使用条件与多测 每次建模须清空流网络、需求平衡数组和下界费用和。上下界可行流阶段结束后，应删除所有超级源汇的关联边及其反向残量，并同时删除额外 $T \to S$ 边的两个方向，再在原源汇上增广；基准流量从删除前的额外边反向残量读取。最小非负流问题退流不能超过基准流量；允许负值流时按原定义处理。INF 应大于可能的总割容量并避免溢出；最长反链模型要求 DAG 所定义的偏序（或先缩 SCC 后按题意赋权），需要覆盖传递可达关系，可用 out 到 in 的原边加 in 到 out 的无穷容量边传递约束。费用版还须累计下界费用，负边饱和转化会改变顶点需求与基准费用。

2.5 图匹配 (Graph Matching)

2.5.1 二分图最小权完美匹配 (KM / SSP)

Kuhn-Munkres (KM) 算法，用于 O(n^3) 求解二分图最小/大权完美匹配。

```
ll cst[1010][1010];
#define inf 0x3f3f3f3f3f3f3f3f
```

```cpp
pair<ll,vector<int>> KM(int n,int m){
    if(n>m)return {inf,{}};
    vector<ll>l(n+1),r(m+1);
    vector<int>p(m+1),ans(n+1);
    f(i,1,n){
        vector<ll>dis(m+1,inf);
        vector<int>pre(m+1),done(m+1);
        int x,v=0,u=0;
        for(p[0]=i;x=p[u];u=v){
            done[u]=1;
            ll mn=inf;
            f(j,1,m)if(!done[j]){
                ll w=cst[x][j]-l[x]-r[j];
                if(w<dis[j])dis[j]=w,pre[j]=u;
                if(dis[j]<mn)mn=dis[j],v=j;
            }
            if(mn==inf)return {inf,{}};
            f(j,0,m){
                if(done[j])l[p[j]]+=mn,r[j]-=mn;
                else dis[j]-=mn;
            }
        }
        for(int v;u;u=v)v=pre[u],p[u]=p[v];
    }f(j,1,m)if(p[j])ans[p[j]]=j;
    return {-r[0],ans};
    // cout<<(-r[0])<<endl;//最小权
    // f(j,1,m)cout<<p[j]<<" ";cout<<endl;//右部点的匹配点。左部点匹配点是 ans。
}
```

稀疏图费用流做法（SSP） 复用前文 MCMF 代码而不是另写一份松弛器。下例为与其放在同一作用域的适配函数，左右点分别从 1 编号；每组重新建残量网络。最大匹配数下求最小费用，要求完美匹配时额外检查返回流量是否等于左部点数。允许原始边负费用；初始网络为 DAG，因此无负费用环。

```cpp
array<ll,2> sparse_matching(int left,int right,
                            const vector<tuple<int,int,ll>>& edges){
    nodes=left+right+2;S=left+right+1;T=S+1;
    assert(nodes<100010&&2*(left+right+edges.size())+2<1000010);
    fill(hd,hd+nodes+1,0);cnt=1;cst=0;
    for(int u=1;u<=left;u++)ad(S,u,1,0);
    for(int v=1;v<=right;v++)ad(left+v,T,1,0);
    for(auto [u,v,c]:edges)ad(u,left+v,1,c);
    spfa(S);return mcmf();
}
```

使用条件与多测 稠密 KM 与稀疏 SSP 为两个接口。第一份返回最小费用及左部匹配，要求 $n \le m$、完整有限费用矩阵；每组必须完整覆盖 cst[1..n][1..m]，其余状态局部化。不可用边不要直接当无限大参与减势运算，稀疏图优先用费用流；若用有限哨兵需证明足够大且最终检查是否选到禁边。矩形复杂度 $O(n^2m)$。SSP 适配器依赖前文 MCMF（去掉其 main 后合用），会重建流网络；费用和减势运算不得溢出，目标是最大匹配数下的最小费用，未必完美匹配。复杂度为初始化加每次增广的 $O(E \log V)$。

2.6 树上算法 (Tree Algorithms)

2.6.1 树的重心 (Tree Centroid)

从点分模板贺的，所以有 fl 数组表示有没有被处理过。

```
ll sz[100010],g,mn;
ll f[100010];
bool fl[100010];
ll siz;
void dfsz(ll u,ll fa){if(fl[u])return;
  f[u]=fa;siz++;
  sz[u]=1;for(int i=hd[u];i;i=e[i].nx){
    ll v=e[i].v,w=e[i].w;
    if(v==fa||fl[v])continue;
    dfsz(v,u),sz[u]+=sz[v];
  }
}
void dfsg(ll u,ll fa){if(fl[u])return;
  ll mx=0;
  if(f[u]!=0)mx=siz-sz[u];
  for(int i=hd[u];i;i=e[i].nx){
    ll v=e[i].v,w=e[i].w;
    if(v==fa||fl[v])continue;
    dfsg(v,u),mx=max(mx,sz[v]);
  }if(mx<mn){mn=mx,g=u;}
}
void getg(ll rt){
  mn=0x7f7f7f7f;siz=0;g=0;
  dfsz(rt,0),dfsg(rt,0);
}
```

使用条件与多测 每组重建邻接表并清 fl[1..n]；同组点分治递归时 fl 是已删重心标记，不能每次 getg 都清空。rt 必须位于当前未删除的树分量；若 fl[rt] 为真返回 g=0。每次 getg 已重置 siz、mn、g，sz/f 在当前分量重新覆盖。深链注意递归栈，单次 O(分量大小)。

2.6.2 树上离某个点最远的点

无权树，down/up 都以边数表示距离，包含自身的距离 0。先 dfs_down(root,0) 再令 up[root]=0 并 dfs_up(root,0)；答案为 max(down[u],up[u])，根的其他子树也会被正确考虑。每组重建邻接表，两个遍历覆盖有效数组。$O(n)$；深链需改为迭代遍历。

```
ll down[200010],second_down[200010],up[200010];
int best_child[200010];
vector<int> e[200010];
void dfs_down(int u,int parent){
    down[u]=second_down[u]=0;best_child[u]=0;
    for(int v:e[u])if(v!=parent){
        dfs_down(v,u);ll x=down[v]+1;
        if(x>down[u])second_down[u]=down[u],down[u]=x,best_child[u]=v;
        else second_down[u]=max(second_down[u],x);
    }
}
void dfs_up(int u,int parent){
    for(int v:e[u])if(v!=parent){
        ll sibling=(best_child[u]==v?second_down[u]:down[u]);
        up[v]=max(up[u],sibling)+1;dfs_up(v,u);
```

```
    }
}
```

2.6.3 斯坦纳树 (Steiner Tree)

无向非负权图上的子集 DP。顶点从 1 编号，终端可重复（会去重）；空终端集返回 0，不可连通返回 INF。局部状态支持多测。多源 Dijkstra 直接在原邻接表运行，省去 Floyd；时间 $O(3^k n + 2^k(m+n)\log n)$，空间 $O(2^k n)$，仅适用于终端数很小的情况。

```
ll steiner_tree(const vector<vector<pair<int,ll>>>& g,vector<int> terminal){
    const ll INF=LLONG_MAX/4;
    int n=int(g.size())-1;
    sort(terminal.begin(),terminal.end());
    terminal.erase(unique(terminal.begin(),terminal.end()),terminal.end());
    int k=terminal.size();if(!k)return 0;
    assert(k<20); // memory, not just shift width, is the real constraint
    int all=1<<k;vector<vector<ll>> dp(all,vector<ll>(n+1,INF));
    for(int i=0;i<k;i++)dp[1<<i][terminal[i]]=0;
    for(int mask=1;mask<all;mask++){
        for(int sub=(mask-1)&mask;sub;sub=(sub-1)&mask){
            int other=mask^sub;if(sub>other)continue;
            for(int u=1;u<=n;u++)dp[mask][u]=min(dp[mask][u],dp[sub][u]+dp[other][u]);
        }
        priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> q;
        for(int u=1;u<=n;u++)if(dp[mask][u]<INF)q.push({dp[mask][u],u});
        while(!q.empty()){
            auto [distance,u]=q.top();q.pop();if(distance!=dp[mask][u])continue;
            for(auto [v,w]:g[u]){
                assert(w>=0&&w<INF);
                if(distance<=INF-w&&distance+w<dp[mask][v])
                    dp[mask][v]=distance+w,q.push({distance+w,v});
            }
        }
    }
    return *min_element(dp.back().begin()+1,dp.back().end());
}
```

2.7 2-SAT

依赖前文 Tarjan SCC。明确约定：顶点 i 表示 $x_i = false$，i+n 表示 $x_i = true$。子句 $(a \vee b)$ 加 $\neg a \to b$ 和 $\neg b \to a$；下例 a,b 为上述文字编号。必须先检查矛盾，再使用逆拓扑 SCC 编号构造赋值。每组调用 Tarjan 的 init(2*n)，重建所有蕴含边。

```
int neg_literal(int a,int n){return a<=n?a+n:a-n;}
void add_clause(int a,int b,int n){
    e[neg_literal(a,n)].push_back(b);
    e[neg_literal(b,n)].push_back(a);
}
optional<vector<int>> solve_2sat(int n){
    for(int u=1;u<=2*n;u++)if(!dfn[u])dfs(u);
    vector<int> value(n+1);
    for(int i=1;i<=n;i++){
        if(bel[i]==bel[i+n])return nullopt;
        value[i]=(bel[i+n]<bel[i]);
    }
    return value;
}
```

3 动态规划 (Dynamic Programming)

3.1 WQS 二分

WQS 二分（Alien Trick）用于“恰好选 m 个”的最优化问题。设原 DP 的一个方案代价为 V，使用了 c 段（或选了 c 个物品），给每一段附加代价 k 后，转而最小化

$$V + kc.$$

当 k 增大时，最优方案使用的段数 $c(k)$ 单调不增，因此可以二分 k，把一维“段数”从 DP 状态中消掉。若目标为恰好 m 段，在边界处有

$$\text{ans} = F_k(n) - km.$$

适用时需要确认两件事：一是加权后的最优段数关于 k 单调；二是要恢复的 (c, V) 点确实位于这些方案的下凸包上，也就是关于段数的最优值具有所需的离散凸性。第二点不能只由“段数单调”推出，需要结合题目的费用结构证明；很多满足四边形不等式的分段 DP 会出现这种结构。整数代价下通常直接二分整数 k；若需要非整数斜率，则按题目精度扩大或改为实数处理。

实现中 DP 同时记录 $(\text{cost}, \text{cnt})$。求最小值时若加权代价相同，通常取更小的 cnt，这样 $c(k)$ 在二分边界处定义稳定。下面是可复用核心：

```
struct State{
    ll v; int cnt;
};
State dp[N];

bool better(State a,State b){
    if(a.v!=b.v)return a.v<b.v;
    return a.cnt<b.cnt;              // min problem: tie -> fewer groups
}

int check(ll k){
    dp[0]={0,0};
    for(int i=1;i<=n;i++){
        dp[i]={INF,0};
        for(int j=0;j<i;j++){
            if(dp[j].v==INF)continue;
            State t={dp[j].v+cost(j+1,i)+k,dp[j].cnt+1};
            if(better(t,dp[i]))dp[i]=t;
        }
    }
    return dp[n].cnt;
}

ll wqs(int m,ll L,ll R){
    while(L<R){
        ll mid=ll((__int128)L+((__int128)R-L)/2);
        if(check(mid)<=m)R=mid;
        else L=mid+1;
    }
    check(L);
    return dp[n].v-L*m;
}
```

上面只展示 WQS 本身，`check` 内部仍是 $O(n^2)$；实际使用时再套斜率优化、决策单调性等优化即可。

使用条件与多测 每次 check 已重建 dp[0..n]，但 cost 依赖的前缀和 / 数据结构必须由调用方按组重建。要求目标段数可行、边界 L,R 能夹住所需斜率、离散凸性成立；整数二分不能用于需要非整数临界斜率的任意问题。所有费用运算及 km 须可表示，必要时整体改用 128 位。

3.2 决策单调性与二分队列

考虑一类一维分段 DP

$$f_i = \min_{0\le j<i}\{f_j + w(j+1,i)\}.$$

若转移矩阵是 Monge 的，即对 $j < J \le i < I$ 有四边形不等式

$$w(j+1,i) + w(J+1,I) \le w(j+1,I) + w(J+1,i),$$

则最优决策点 $p_i = \operatorname{argmin}_j\{f_j + w(j+1,i)\}$ 单调不减。加上只与 j 有关的 f_j 不会破坏 Monge 性。

对区间费用 $C(l,r)$，常写成

$$C(a,c) + C(b,d) \le C(a,d) + C(b,c), \qquad a \le b \le c \le d.$$

在完整的二维定义域上，也可以只检查相邻的 2×2 条件

$$C(l,r) + C(l+1,r+1) \le C(l,r+1) + C(l+1,r),$$

它等价于所有四边形不等式。实际题目中通常直接从费用的结构证明；例如有序点分段后到中位数的 L_1 代价就是常见的 Monge 费用。

二分队列维护若干候选决策 (j, L)，表示从位置 L 开始，j 是当前队列中最优的决策。由于决策单调，两个候选的优劣只会发生一次切换：插入新决策 i 时，与队尾旧决策比较，并二分出新决策第一次不劣的位置；若新决策从队尾的起点起就更优，则直接弹掉队尾。查询 f_i 时只需根据这些分界点移动队首。每个决策至多入队、出队一次，另有一次二分，因此总复杂度 $O(n\log n)$。

下面是最小化问题的核心模板。`better(j,J,x)` 表示在终点 x 处决策 j 是否严格优于 J；相等时的 tie-break 要与 WQS 中记录的段数保持一致。

```
ll dp[N];
int cnt[N];
struct Node{ int j,l; } q[N];
int hd,tl;

bool better(int j,int J,int i){
    ll x=dp[j]+cost(j+1,i);
    ll y=dp[J]+cost(J+1,i);
    if(x!=y)return x<y;
    return cnt[j]<cnt[J];
}

void take(int j,int i,ll extra=0){
    dp[i]=dp[j]+cost(j+1,i)+extra;
    cnt[i]=cnt[j]+1;
}

void insert_candidate(int i){
    if(i>=n)return;
    while(hd<=tl){
        int j=q[tl].j;
        int l=max(i+1,q[tl].l),r=n;

```

```
        // old candidate is still better at n: i never becomes optimal
        if(better(j,i,n))return;

        // first position where i is no worse than j
        while(l<r){
            int mid=(l+r)>>1;
            if(better(j,i,mid))l=mid+1;
            else r=mid;
        }

        if(l==q[tl].l)--tl;          // i dominates the whole last interval
        else{
            q[++tl]={i,l};
            return;
        }
    }
    q[++tl]={i,i+1};
}

void solve_dp(ll extra=0){
    dp[0]=0;cnt[0]=0;
    hd=tl=1;q[1]={0,1};
    for(int i=1;i<=n;i++){
        while(hd<tl&&q[hd+1].l<=i)++hd;
        take(q[hd].j,i,extra);
        insert_candidate(i);
    }
}
```

和 WQS 合用时，令 `extra=k`，一次 `check(k)` 就是一次 $O(n \log n)$ 的上述 DP，于是总复杂度再乘一个 WQS 的二分次数。

使用条件与多测 solve_dp 每次重置 dp[0]、cnt[0]、队列边界；其余 dp/cnt 在使用前被顺序覆盖。cost 的预处理由调用方重建。Monge 条件须证明且两候选含 tie-break 的比较须单次切换；和 WQS 组合时也应验证段数 tie-break 满足该条件，不能只验证数值最优点单调。要求 cost 为 $O(1)$ 才有所述复杂度；中间加法须防溢出。

4 字符串 (String Algorithms)

4.1 字符串哈希 (String Hashing)

用于 O(n) 预处理，O(1) 查询任意子串的哈希值。这个板子好像挺容易冲突的，建议双 hash。

```
const ll mod=180181327,bs=31;
ll pw[1000010];
ll h[1000010],n;
char c[1000010];
ll get(ll l,ll r){
    return (h[r]-pw[r-l+1]*h[l-1]%mod+mod)%mod;
}
int main(){
    pw[0]=1;f(i,1,1000000)pw[i]=pw[i-1]*bs%mod;
    scanf("%1000000s",c+1);n=strlen(c+1);h[0]=0;
    f(i,1,n)h[i]=(h[i-1]*bs+(c[i]-'a'+1))%mod;
}
```

使用条件与多测 幂表固定底数 / 模数时仅预处理一次；每组重建 h[0..n]。本实现只适用于小写字母，查询非空且边界合法。哈希相等并非严格相等，建议独立双模数随机底数，或在要求确定正确时使用后缀数组 / 直接校验。

4.2 KMP

Knuth-Morris-Pratt 算法，用于在线性时间内进行字符串匹配（在 c 串中找 s 串）。

```
char c[1000010],s[1000010];
ll nx[1000010];
int main(){
  cin>>setw(1000001)>>(c+1)>>setw(1000001)>>(s+1);
  n=strlen(c+1),m=strlen(s+1);
  nx[0]=nx[1]=0;
    ll j=0;f(i,2,m){
    while(j&&s[j+1]!=s[i])j=nx[j];
    if(s[j+1]==s[i])j++;nx[i]=j;
  }j=0;f(i,1,n){
    while(j&&s[j+1]!=c[i])j=nx[j];
    if(s[j+1]==c[i])j++;
    if(j==m){cout<<i-m+1<<'\n';j=nx[j];}
  }
}
```

使用条件与多测 每组 nx[1]=0、j=0，当前有效 nx 区间会覆盖；主串和模式串需重新读入。模式串必须非空，空模式须另行定义。已限制 char 数组输入宽度，仍要求单串不超过 1000000 字符；C++20 下建议改用 string（原 char 指针输入惯用法用 C++17 编译）。

4.3 Manacher

返回最长回文子串长度，空串返回 0。分隔符使用整数 256，与任意字节均不同；每次调用全部状态为局部变量，支持直接多测。时间、空间均为 $O(n)$。

```
int manacher(const string& s){
    vector<int> a(1,256);
    for(unsigned char c:s){a.push_back(c);a.push_back(256);}
    int n=a.size(),mid=0,right=-1,ans=0;
    vector<int> radius(n);
    for(int i=0;i<n;i++){
        if(i<=right)radius[i]=min(right-i,radius[2*mid-i]);
        while(i-radius[i]-1>=0&&i+radius[i]+1<n&&
              a[i-radius[i]-1]==a[i+radius[i]+1])radius[i]++;
        if(i+radius[i]>right)mid=i,right=i+radius[i];
        ans=max(ans,radius[i]);
    }
    return ans;
}
```

4.4 AC 自动机 (Aho-Corasick)

小写字母稠密转移表。插入返回终点编号，支持重复模式；match 返回每个节点所代表字符串在文本中的出现次数。每组先 clear，插完全部模式后 build，之后才可匹配。构建 $O(26V)$，单次匹配 $O(|text|+V)$。

```cpp
struct AhoCorasick {
    struct Node{array<int,26> next{};int fail=0;};
    vector<Node> t;
    vector<int> order;
    bool built=false;
    AhoCorasick(){clear();}
    void clear(){t.assign(1,{});order.clear();built=false;}
    int add(const string& s){
        assert(!built&&!s.empty());int u=0;
        for(char c:s){
            int x=c-'a';assert(0<=x&&x<26);
            if(!t[u].next[x]){
                int v=t.size();t.push_back({});t[u].next[x]=v;
            }
            u=t[u].next[x];
        }
        return u;
    }
    void build(){
        assert(!built);queue<int> q;
        for(int v:t[0].next)if(v)q.push(v);
        while(!q.empty()){
            int u=q.front();q.pop();order.push_back(u);
            for(int c=0;c<26;c++){
                int v=t[u].next[c],f=t[u].fail;
                if(v)t[v].fail=t[f].next[c],q.push(v);
                else t[u].next[c]=t[f].next[c];
            }
        }
        built=true;
    }
    vector<ll> match(const string& s)const{
        assert(built);vector<ll> count(t.size());int u=0;
        for(char c:s){int x=c-'a';assert(0<=x&&x<26);u=t[u].next[x];count[u]++;}
        for(int i=int(order.size())-1;i>=0;i--){
            int v=order[i];count[t[v].fail]+=count[v];
        }
        return count;
    }
};
```

使用条件与多测 build 会补全所有转移，不能再次 build 或继续 add；下一组必须 clear。空模式未定义且禁止插入。稀疏字符集版本需独立使用真实 trie 边，根的 fail 为 0 或显式跳根特判，不能直接沿用本实现补全后的转移或把 fail[root] 设为 root 后无条件跳 fail，否则可能死循环。

4.5 后缀数组 (Suffix Array - SA)

倍增加两趟计数排序，时间 $O(n \log n)$、空间 $O(n)$；支持任意字节和空串。sa 为 0 下标后缀起点，rank 为 0 下标排名，lcp[i] 是 sa[i] 与 sa[i-1] 的 LCP，lcp[0]=0。所有状态在每次 build 中重建。

```cpp
struct SuffixArray {
    vector<int> sa,rank,lcp;
    void build(const string& s){
        int n=s.size();sa.resize(n);rank.resize(n);lcp.assign(n,0);
        if(!n)return;
        vector<int> x(n),y(n),tmp(n),cnt(max(n,256)+1);
        for(int i=0;i<n;i++)x[i]=(unsigned char)s[i]+1,sa[i]=i;
```

```
        auto sort_by=[&](const vector<int>& key,int m){
            fill(cnt.begin(),cnt.begin()+m+1,0);
            for(int id:sa)cnt[key[id]]++;
            for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
            for(int i=n-1;i>=0;i--)tmp[--cnt[key[sa[i]]]]=sa[i];
            sa.swap(tmp);
        };
        int m=256;
        for(int k=1;;k*=2){
            for(int i=0;i<n;i++)y[i]=(i+k<n?x[i+k]:0);
            sort_by(y,m);sort_by(x,m);
            int classes=1;y[sa[0]]=classes;
            for(int i=1;i<n;i++){
                int a=sa[i-1],b=sa[i];
                if(x[a]!=x[b]||(a+k<n?x[a+k]:0)!=(b+k<n?x[b+k]:0))classes++;
                y[b]=classes;
            }
            x.swap(y);m=classes;if(m==n)break;
        }
        for(int i=0;i<n;i++)rank[sa[i]]=i;
        for(int i=0,h=0;i<n;i++){
            int r=rank[i];if(!r){h=0;continue;}
            int j=sa[r-1];
            while(i+h<n&&j+h<n&&s[i+h]==s[j+h])h++;
            lcp[r]=h;if(h)h--;
        }
    }
};
```

4.6 回文自动机 (Palindrome Automaton - PAM)

小写字母回文自动机，add 返回以当前字符结尾的不同回文长度数量（即当前回文后缀数）；不含原题特定的输入字符加密。每组 clear；总时间 $O(26n)$（固定字符集视作线性）、空间 $O(26n)$。

```
struct PalindromicTree {
    struct Node{array<int,26> next{};int len=0,fail=0,suffixes=0;ll occ=0;};
    vector<Node> t;
    vector<int> text;
    int last=0;
    PalindromicTree(){clear();}
    void clear(){
        t.assign(2,{});t[0].len=0;t[0].fail=1;
        t[1].len=-1;t[1].fail=1;text.assign(1,-1);last=0;
    }
    int getfail(int u){
        int pos=int(text.size())-1;
        while(text[pos-t[u].len-1]!=text[pos])u=t[u].fail;
        return u;
    }
    int add(char c){
        int x=c-'a';assert(0<=x&&x<26);text.push_back(x);
        int u=getfail(last);
        if(!t[u].next[x]){
            int v=t.size();t.push_back({});t[v].len=t[u].len+2;
            t[v].fail=(t[v].len==1?0:t[getfail(t[u].fail)].next[x]);
            t[v].suffixes=t[t[v].fail].suffixes+1;t[u].next[x]=v;
        }
        last=t[u].next[x];t[last].occ++;
        return t[last].suffixes;
```

```
    }
    vector<ll> occurrences()const{ // non-destructive; safe to call repeatedly
        vector<ll> count(t.size());
        for(int i=2;i<int(t.size());i++)count[i]=t[i].occ;
        for(int i=int(t.size())-1;i>=2;i--)count[t[i].fail]+=count[i];
        return count;
    }
};
```

5 数学 (Mathematics)

5.1 数论 (Number Theory)

5.1.1 Exgcd (Extended Euclidean Algorithm)

返回 $g = \gcd(a, b)$ 和满足 $ax + by = g$ 的系数；先判断 $g \mid c$，再将系数乘以 c/g 才得到方程 $ax + by = c$ 的解。避免原版直接 c/a 截断而默默给出伪解。参数为非负 ll 且不同时为 0；负数需先取绝对值并恢复系数符号，LLONG_MIN 需先转 128 位。无全局状态，可直接多测。

```
ll exgcd(ll a,ll b,ll &x,ll &y){
    assert(a>=0&&b>=0&&(a||b));
    if(!b){x=1;y=0;return a;}
    ll u,v,g=exgcd(b,a%b,u,v);
    x=v;y=ll((__int128)u-(__int128)(a/b)*v);return g;
}
bool diophantine(ll a,ll b,ll c,__int128_t &x,__int128_t &y){
    if(!a&&!b){x=y=0;return c==0;}
    ll u,v,g=exgcd(a,b,u,v);if(c%g)return false;
    x=(__int128)u*(c/g);y=(__int128)v*(c/g);return true;
}
```

5.1.2 万能欧几里得

把 linear 看成 U（穿过横轴）和 R（穿过纵轴）的操作序列，复合操作来优化复杂度。

```
//Operation.
//Cross a horizontal line => U operation (impact of floor sum+1)
//Cross a vertical line => R operation (impact of i+1)
struct node{
    ll s,numU,numR;
    node(ll s=0,ll numU=0,ll numR=0){
        this->s=s,this->numU=numU,this->numR=numR;
    }
    node operator + (const node& b){
        node c;
        c.numR=numR+b.numR;
        c.numU=numU+b.numU;
        c.s=s+b.s+numU*b.numR;
        return c;
    }
};
//pow of operation
node opow(node a,ll b){
    node res;
    while(b){
        if(b&1)res=res+a;
        b>>=1,a=a+a;
    }return res;
}
// need a>=0, b>=0, b<m
// 计算 [1,n] 的UR复合操作的结果
node work(ll a,ll b,ll n,ll m,node U,node R){
    if(!n)return node();//构造函数应该取单位元，若没有单位元则需要小心处理opow里b=0和本行的情况。
    if(a>=m)return work(a%m,b,n,m,U,opow(U,a/m)+R);
    ll cu=(ll)(((__int128)n*a+b)/m); // must fit ll exactly
    if(!cu)return opow(R,n);
    ll cnt=n-(ll)(((__int128)m*cu-b-1)/a);
```

```
    return opow(R,(m-b-1)/a)+U+work(m,(m-b-1)%a,cu-1,a,R,U)+opow(R,cnt);
}
node U,R;

// 使用例: \sum_{i=0}^{n-1} \lfloor\frac{ai+b}{m}\rfloor
// for b>=m, use opow(U,b/m)+work(a,b%m,n,m,U,R) to obtain the result.
void solve(){
  U=(node){0,1,0},R=(node){0,0,1};
    n=d,m=d;ll a=d,b=d;
    if(n==0){puts("0");return;}--n;
    node res=opow(U,b/m)+work(a,b%m,n,m,U,R);//b>=m的时候要线提出几个 U 来用。
    printf("%lld\n",res.s+b/m);
    // out(res.s+b/m);puts("");
}
```

使用条件与多测 要求 $n \geq 0, m > 0, a \geq 0, b \geq 0$，work 的参数还要求 $b < m$；每次 solve 都重新构造 U、R，状态不跨组保存。中间整除必须用精确整数，不可用 long double 替代；cu、复合操作 numU/numR/s 的最终范围仍须保证在 ll 内，较大 floor sum 应将 node 的数值整体改为 128 位。时间 $O(\log m \log(n + a + 1))$ 的保守界，取决于操作快速幂。

5.1.3 ExCRT (Extended Chinese Remainder Theorem)

合并 $x \equiv A \pmod{B}$ 与 $x \equiv a \pmod{b}$。返回 false 表示无解且不修改 A、B；模数必须为正，合并后的最小公倍数超过 ll 时抛异常。内部 128 位计算，避免原版解系数乘积溢出。每组从 A=0、B=1 开始；无解后本组剩余约束仍需读完。

```
using i128=__int128_t;
i128 egcd128(i128 a,i128 b,i128& x,i128& y){
    if(!b){x=1;y=0;return a;}
    i128 u,v,g=egcd128(b,a%b,u,v);x=v;y=u-a/b*v;return g;
}
bool merge(ll &A,ll &B,ll a,ll b){
    assert(B>0&&b>0);
    i128 aa=((i128)A%B+B)%B,bb=((i128)a%b+b)%b;
    i128 x,y,g=egcd128(B,b,x,y),diff=bb-aa;
    if(diff%g)return false;
    i128 period=(i128)B/g*b;
    if(period>LLONG_MAX)throw overflow_error("CRT modulus overflow");
    i128 step=(diff/g*x)%(b/g);
    i128 answer=(aa+(i128)B*step)%period;
    if(answer<0)answer+=period;
    A=(ll)answer;B=(ll)period;return true;
}
```

5.1.4 扩展欧拉定理 (Euler's Totient Theorem)

扩展欧拉定理，用于处理 b 很大的幂取模。当 $a, m \in \mathbb{Z}$ 时有：

$$a^b \equiv \begin{cases} a^b, & b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)}, & b \geq \varphi(m) \end{cases} \pmod{m}$$

使用条件与多测 要求模数 $m > 0$、指数 $b \geq 0$；$m = 1$ 结果恒为 0，通常约定 $0^0 = 1$ 后再取模。必须同时维护“原指数是否达到 $\varphi(m)$”的信息，不能只保存取模后的指数。$\varphi(m)$ 的缓存仅在模数相同时可复用；变模数须重算。

5.1.5 Pollard-Rho

Pollard's Rho 算法，一种用于大整数质因数分解的随机算法。

```
ll qp(ll a,ll b,ll p){a%=p;
  ll ans=1;while(b){
    if(b&1)ans=(__int128)ans*a%p;
    a=(__int128)a*a%p;b>>=1;
  }return ans;
}ll n,m;
ll tst[7]={2,325,9375,28178,450775,9780504,1795265022};
/*
unsigned : 2 7 61
ull : 2 325 9375 28178 450775 9780504 1795265022
*/
bool miller(ll x){if(x<2)return 0;
    if(x==2||x==3||x==5||x==7||x==11||x==13)return 1;
  ll u=x-1,t=0;while(u%2==0)u/=2,t++;
  f(T,0,6){
    ll a=tst[T];
    if(a%x==0)continue;
    ll b=qp(a,u,x),bb;//if(b==1||b==x-1)continue;
    f(i,1,t){
      bb=(__int128)b*b%x;
      if(bb==1&&b!=1&&b!=x-1)return 0;b=bb;
    }if(b!=1)return 0;
  }return 1;
}ll pollard(ll p){
  if(p%2==0)return 2;
    static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    ll c=uniform_int_distribution<ll>(1,p-1)(rng);
  ll s=0,t=0,mul=1,k=1,i=0;
  for(k=1;k<=(1LL<<20);k<<=1,s=t,mul=1){
    for(i=1;i<=k;i++){
      t=((__int128)t*t+c)%p;
      mul=(__int128)mul*abs(s-t)%p;
      if(i%127==0){
        ll g=__gcd(mul,p);
        if(g>1)return g;
      }
    }ll g=__gcd(mul,p);
    if(g>1)return g;
  }
    return p; // restart with a new random polynomial
}ll mxp;
void work(ll p){
  if(p<2||p<=mxp)return;
  if(miller(p)){mxp=max(mxp,p);return;}
  ll k=p;while(k==p)k=pollard(p);
  while(p%k==0)p/=k;
  work(p),work(k);
}
int main(){srand(time(0));
  wt{ ll p=d;mxp=0;
    work(p);if(mxp==p)puts("Prime");
    else printf("%lld\n",mxp);
  }
  return 0;
}
```

使用条件与多测 本版本输入为正 signed long long，Miller–Rabin 的七基数覆盖该范围；底数先模 n，模 n 为 0 时须跳过，已修复会把 73 等质数误判的情况。每个输入将 mxp=0；随机数引擎跨组保留。work 只求最大质因子，不返回完整分解；$n = 1$ 无质因子，应由题意单独处理。Pollard–Rho 为随机期望算法，无固定最坏时限；不要把此 ll 接口直接用于完整 unsigned 64 位范围。

5.1.6 Dirichlet 前缀和 (Dirichlet Prefix Sum)

Dirichlet 前缀和，用于快速计算形如 $f(i) = \sum_{x|i} a_x$ 的函数。

$$a_i \to \sum_{x|i} a_x$$

```
for(i=1;i<=cnt;++i)for(j=1;j*pr[i]<=n;++j)a[j*pr[i]]+=a[j];
```

使用条件与多测 pr 必须是从小到大不重复的素数表，cnt 为有效素数个数；代码原地执行约数和变换，每组先把 a[1..n] 恢复为原始输入，不能对已变换结果重复调用。循环 j 必须递增；逆变换则反序执行减法。约 $O(n \log\log n)$，模意义下每步取模，整数求和须避免溢出。

5.1.7 杜教筛 (Du's Sieve)

杜教筛，用于在亚线性时间内求解数论函数（如 μ, ϕ）的前缀和。设 $S(n) = \sum_{i=1}^{n} f(i)$，我们构造 g 使得 $\sum(f * g)$ 和 $\sum g$ 易于计算。其核心公式为：

$$g(1)S(n) = \sum_{i=1}^{n}(f * g)(i) - \sum_{i=2}^{n} g(i)S(\lfloor n/i \rfloor)$$

```
ll n,m;
ll pr[2000010];
bool vis[2000010];
ll mu[2000010];
ll ph[2000010];
void gt(){
    static bool ready=false;if(ready)return;ready=true;cnt=0;
  mu[1]=ph[1]=1;
  f(i,2,2000000){
    if(!vis[i])pr[++cnt]=i,mu[i]=-1,ph[i]=i-1;
    for(int j=1;j<=cnt&&i*pr[j]<=2000000;j++){
      vis[i*pr[j]]=1;if(i%pr[j]==0){
        ph[i*pr[j]]=ph[i]*pr[j];break;
      }mu[i*pr[j]]=-mu[i],ph[i*pr[j]]=ph[i]*ph[pr[j]];
    }
  }f(i,2,2000000)mu[i]+=mu[i-1],ph[i]+=ph[i-1];
}
unordered_map<ll,ll>smu;
unordered_map<ll,ll>sph;
ll Mu(ll n){
  if(n<=2000000)return mu[n];
  if(auto it=smu.find(n);it!=smu.end())return it->second;
  ll res=1;for(ll l=2,r;l<=n;l=r+1){
    r=n/(n/l);res-=(r-l+1)*Mu(n/l);
  }return smu[n]=res;
}ll Ph(ll n){
  if(n<=2000000)return ph[n];
  if(auto it=sph.find(n);it!=sph.end())return it->second;
```

```
  ll res=(ll)((__int128)n*(n+1)/2);for(ll l=2,r;l<=n;l=r+1){
    r=n/(n/l);res-=(r-l+1)*Ph(n/l);
  }return sph[n]=res;
}
int main(){
  gt();wt{
    n=d;printf("%lld %lld\n",Ph(n),Mu(n));
  }
  return 0;
}
```

使用条件与多测　固定函数和固定筛上限时 gt 只调用一次，前缀表和两个 memo 均可跨组复用；如受内存限制可在组间清 smu、sph（影响速度但不影响结果）。零值缓存必须用 find 判存在，已修复反复计算和查询插入空缓存的问题。输入非负，$\sum\phi$ 及中间项须在 ll 范围内；超范围时缓存与运算整体改用 128 位。

5.1.8 Min25

Min25 示例：积性函数满足 $f(p^e)=p^e(p^e-1)$，求 $\sum_{i=1}^{n} f(i)$ 模 10^9+7。一般函数需要替换质数处点值和及素数幂处取值，不能只把一般积性函数拆成完全积性函数后直接组合答案。定义 $S(m,j)$ 为不含前 j 个素因子的 $2..m$ 的函数和，最终答案是 $S(n,0)+1$。数组按 $\sqrt{n}$ 和实际商块数动态分配，修复原固定容量无法覆盖所声称 10^{11} 范围的问题。每次 sum(n) 重新初始化；时间为 Min25 筛及素数幂递归的实际开销，不承诺固定秒数。

```
struct Min25 {
    static constexpr ll mod=1000000007;
    ll n;int lim;
    vector<int> primes,small,large;
    vector<ll> values,g1,g2,s1,s2;
    int index(ll x)const{return x<=lim?small[x]:large[n/x];}
    ll power_value(ll x)const{x%=mod;return x*((x-1+mod)%mod)%mod;}
    ll S(ll m,int j){ // exclude the first j primes
        if(m<2||(j&&primes[j-1]>=m))return 0;
        int id=index(m);
        ll ans=(g2[id]-g1[id]-s2[j]+s1[j])%mod;
        if(ans<0)ans+=mod;
        for(int i=j;i<int(primes.size())&&ll(primes[i])*primes[i]<=m;i++){
            ll p=primes[i],v=p;int exponent=1;
            for(;;){
                ans=(ans+power_value(v)*(S(m/v,i+1)+(exponent>1)))%mod;
                if(v>m/p)break;v*=p;exponent++;
            }
        }
        return ans;
    }
    ll sum(ll input){
        assert(input>=0&&input<=1000000000000LL);
        if(input<1)return 0;n=input;lim=sqrtl(n);
        while(ll(lim+1)<=n/(lim+1))lim++;
        while(ll(lim)>n/lim)lim--;
        primes.clear();vector<char> composite(lim+1);
        for(int i=2;i<=lim;i++){
            if(!composite[i])primes.push_back(i);
            for(int p:primes){if(ll(i)*p>lim)break;composite[i*p]=true;if(i%p==0)break;}
        }
        s1.assign(primes.size()+1,0);s2=s1;
```

```
        for(int i=0;i<int(primes.size());i++){
            ll p=primes[i];s1[i+1]=(s1[i]+p)%mod;s2[i+1]=(s2[i]+p*p)%mod;
        }
        small.assign(lim+1,0);large=small;values.clear();g1.clear();g2.clear();
        for(ll l=1,r;l<=n;l=r+1){
            ll v=n/l;r=n/v;int id=values.size();values.push_back(v);
            if(v<=lim)small[v]=id;else large[n/v]=id;
            ll x=v%mod;
            g1.push_back((x*(x+1)%mod*500000004%mod+mod-1)%mod);
            g2.push_back((x*(x+1)%mod*((2*x+1)%mod)%mod*166666668%mod+mod-1)%mod);
            if(r==n)break;
        }
        for(int i=0;i<int(primes.size());i++){
            ll p=primes[i];
            for(int j=0;j<int(values.size())&&p*p<=values[j];j++){
                int k=index(values[j]/p);
                g1[j]=(g1[j]-p*((g1[k]-s1[i]+mod)%mod)%mod+mod)%mod;
                g2[j]=(g2[j]-p*p%mod*((g2[k]-s2[i]+mod)%mod)%mod+mod)%mod;
            }
        }
        return (S(n,0)+1)%mod;
    }
};
```

使用条件与多测 本实现范围 $0 \le n \le 10^{11}$，仍须按实际内存和时限判断是否可行，$O(\sqrt{n})$ 存储；不是针对完整 64 位区间设计的接口。素数前缀表可针对固定最大上界复用，但本实现选择逐次重建以明确多测语义。

5.1.9 单位根反演

就一个式子。

$$[n \mid m] = \frac{1}{n}\sum_{i=0}^{n-1}\omega_n^{im}$$

还有一个进阶式子。

$$[m \bmod n = k] = [n|(m-k)] = \frac{1}{n}\sum_{i=0}^{n-1}\omega_n^{im}\omega_n^{-ik}$$

模意义下 $\omega_n = g^{\frac{p-1}{n}}$

使用条件与多测 模域中要求 p 为质数、$n > 0$ 且 $n \mid (p-1)$，g 为原根，因而 n 可逆且根具有恰好 n 阶；不能把任意 n 或非本原根代入。指数负数按 $p-1$ 归一化；更换 p/n 时须重算根和逆元。

5.2 组合数学 (Combinatorics)

5.2.1 组合数 (Binomial Coefficients)

预处理阶乘及其逆元，O(1) 查询组合数。

```
namespace binom{
  const ll Lim=300010,mod=998244353;
  ll jc[Lim],inv[Lim],inc[Lim];
  void pre(){
```

```
        static bool ready=false;if(ready)return;ready=true;
    jc[0]=jc[1]=inc[0]=inc[1]=inv[0]=inv[1]=1;
    f(i,2,Lim-1)jc[i]=jc[i-1]*i%mod,inv[i]=(mod-mod/i)*inv[mod%i]%mod,
    inc[i]=inc[i-1]*inv[i]%mod;
  }ll C(ll n,ll m){if(n<0||m<0||n<m)return 0;assert(n<Lim);return jc[n]*inc[m]%mod*inc[n-m]%mod;}
}
```

使用条件与多测 pre 在第一次查询前调用一次即可跨组复用；要求素数 mod 且预处理上限小于 mod。查询 $n \geq Lim$ 会越界，已增加断言；大 n 改用 Lucas。变更模数或上限时须重建预处理，不能复用旧逆元。

5.2.2 Lucas 定理 (Lucas's Theorem)

Lucas 定理，用于在 O(p $\log_p n$) 时间内求解模小质数 p 的大组合数。（除法为下取整，要求 p 为质数）

$$\binom{n}{m} \equiv \binom{n \bmod p}{m \bmod p}\binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor} \pmod p$$

使用条件与多测 固定小质数 p 时预处理 0..p-1 的阶乘、逆阶乘共 $O(p)$，之后单次 Lucas 为 $O(\log_p n)$；不应每一位都重新 $O(p)$ 计算。每组 p 相同可复用，不同则重建；$n, m \geq 0$ 且 $m > n$ 返回 0。p 很大时该预处理不适用。

5.2.3 ExLucas (Extended Lucas Theorem)

扩展 Lucas：将正模数分解为互质素数幂，去除阶乘中的 p 因子后做 CRT。处理残余大质因子，$P = 1$ 和 $m > n$，修复原版未合并剩余因子的错误。此直接预处理实现限定 $1 \leq P \leq 2 \times 10^6$，单次预处理 $O(P + \sqrt{P})$，查询递归约 $O(\log^2 n)$；大模数必须换用更适合题目约束的算法，不能假设 ExLucas 对任意大 P 都高效。所有状态局部化，可直接多测。

```
ll exlucas(ll n,ll m,ll mod){
    assert(n>=0&&mod>=1&&mod<=2000000);
    if(m<0||m>n||mod==1)return 0;
    auto powmod=[](ll a,ll e,ll p){ll r=1%p;for(;e;e>>=1,a=a*a%p)if(e&1)r=r*a%p;return r;};
    auto inverse=[](ll a,ll p){
        ll b=p,x=1,y=0;
        while(b){ll q=a/b;ll t=a-q*b;a=b;b=t;t=x-q*y;x=y;y=t;}
        assert(a==1);return (x%p+p)%p;
    };
    vector<pair<ll,ll>> factors;ll rest=mod;
    for(ll p=2;p<=rest/p;p++)if(rest%p==0){
        ll pk=1;while(rest%p==0)rest/=p,pk*=p;factors.push_back({p,pk});
    }
    if(rest>1)factors.push_back({rest,rest});
    ll answer=0,period=1;
    for(auto [p,pk]:factors){
        vector<ll> pref(pk+1,1);
        for(ll i=1;i<=pk;i++)pref[i]=pref[i-1]*(i%p?i:1)%pk;
        function<ll(ll)> fact=[&](ll x)->ll{
            if(!x)return 1;
            return fact(x/p)*powmod(pref[pk],x/pk,pk)%pk*pref[x%pk]%pk;
        };
        auto count=[&](ll x){ll r=0;while(x)x/=p,r+=x;return r;};
        ll exponent=count(n)-count(m)-count(n-m);
        ll residue=fact(n)*inverse(fact(m),pk)%pk*inverse(fact(n-m),pk)%pk;
```

```
        residue=residue*powmod(p,exponent,pk)%pk;
        ll step=((residue-answer)%pk+pk)%pk*inverse(period%pk,pk)%pk;
        answer+=period*step;period*=pk;
    }
    return answer;
}
```

5.2.4 min-max 容斥

$$\text{k-th}\max(S)=\sum_{\emptyset\neq T\subseteq S,|T|\geq k}(-1)^{|T|-k}\binom{|T|-1}{k-1}\min(T)$$

$$\text{k-th}\min(S)=\sum_{\emptyset\neq T\subseteq S,|T|\geq k}(-1)^{|T|-k}\binom{|T|-1}{k-1}\max(T)$$

使用条件与多测 要求 $1\leq k\leq|S|$，按多重集合的元素位置计子集，重复值也成立。直接枚举指数复杂度；模运算中二项式系数需满足对应算法条件。没有可变全局状态，多测仅需清调用方的子集 DP / 组合预处理（若参数改变）。

5.2.5 反演

1. 二项式反演

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i)\iff g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$

$$f(k)=\sum_{i=k}^{n}\binom{i}{k}g(i)\iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\binom{i}{k}f(i)$$

2. 子集反演

$$f(S)=\sum_{T\subseteq S}g(T)\iff g(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}f(T)$$

3. 斯特林反演

$$f(n)=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}g(k)\iff g(n)=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}f(k)$$

$$f(k)=\sum_{i=k}^{n}\begin{Bmatrix}i\\k\end{Bmatrix}g(i)\iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\begin{bmatrix}i\\k\end{bmatrix}f(i)$$

其中，

$$\begin{bmatrix}n\\k\end{bmatrix}=(n-1)\begin{bmatrix}n-1\\k\end{bmatrix}+\begin{bmatrix}n-1\\k-1\end{bmatrix}$$

$$\begin{Bmatrix}n\\k\end{Bmatrix}=k\begin{Bmatrix}n-1\\k\end{Bmatrix}+\begin{Bmatrix}n-1\\k-1\end{Bmatrix}$$

边界条件：$S(n,n)=1,S(n,0)=0(n>0)$，两类都是。

使用条件与多测 两类 Stirling 数都有 $S(0,0)=1$；$n>0$ 时 $S(n,0)=0$、$S(n,n)=1$，越界项为 0。第一类在这里是无符号版本，因此反演式中的 $(-1)^{n-k}$ 不能省略。固定上限与模数时预处理表可跨组复用，滚动数组每轮和每组须清目标行。

5.3 线性代数 (Linear Algebra)

5.3.1 高斯消元 (Gaussian Elimination)

高斯消元法，用于求解线性方程组。

```
double a[110][110];
double b[110];
ll p[110];
const double eps=1e-7;
int main(){
  n=d;f(i,1,n){
    f(j,1,n+1)cin>>a[i][j];
  }ll row=1;
    f(i,1,n){
    ll pos=row;f(j,row,n)if(fabs(a[j][i])>fabs(a[pos][i]))pos=j;
        if(row>n||fabs(a[pos][i])<eps)continue;
    swap(a[row],a[pos]);
    double x=a[row][i];f(j,1,n+1)a[row][j]/=x;
    f(j,row+1,n){
      x=a[j][i];
      f(k,i,n+1)a[j][k]-=x*a[row][k];
    }row++;
  }
    // f(i,1,n){
    //     f(j,1,n+1)cout<<a[i][j]<<" ";cout<<endl;
    // }
    f(i,1,n){
        bool fl=1;
        f(j,1,n)if(fabs(a[i][j])>eps)fl=0;
        if(fl){if(fabs(a[i][n+1])>eps){puts("-1");return 0;}}//无解
    }
    f(i,1,n){
        bool fl=1;
        f(j,1,n)if(fabs(a[i][j])>eps)fl=0;
        if(fl){if(fabs(a[i][n+1])<=eps){puts("0");return 0;}}//无限解
    }
    for(int i=n;i>=1;i--){
    b[i]=a[i][n+1];
    f(j,1,i-1)a[j][n+1]-=a[i][n+1]*a[j][i];
  }
    f(i,1,n)printf("x%lld=%.2lf\n",(long long)i,b[i]);
  return 0;
}
```

使用条件与多测 已改为选绝对值最大主元以减轻数值误差。每组完整重读增广矩阵 a[1..n][1..n+1]，row=1，结果 b 会重写；消元破坏原矩阵，重复求解须先复制。增广矩阵已改用 double 输入，避免整数快读截断实数；eps 应按尺度调整，病态系统不能靠固定绝对阈值保证精度。时间 $O(n^3)$，本代码为方阵分类，无穷解仅输出标识。

5.3.2 行列式

如果模数不是质数，需要对行的主元进行辗转相除。

```
ll qp(ll a,ll b){
    ll res=1;
    while(b){
        if(b&1)res=res*a%mod;
        a=a*a%mod,b>>=1;
    }return res;
}
ll det(ll n,ll a[][1010]){
    int sgn=1;
    f(i,1,n)f(j,1,n)a[i][j]=(a[i][j]%mod+mod)%mod;
    f(i,1,n){
        int p=-1;
        f(j,i,n)if(a[j][i]!=0){p=j;break;}
        if(p==-1){return 0;}
        if(p!=i)swap(a[i],a[p]),sgn=-sgn;
        ll iv=qp(a[i][i],mod-2);
        f(j,i+1,n){
            ll o=a[j][i]*iv%mod;
            f(k,i,n)a[j][k]=(a[j][k]-a[i][k]*o%mod+mod)%mod;
        }
    }
    ll res=1;
    f(i,1,n)res=res*a[i][i]%mod;
    if(sgn==-1)res=(mod-res)%mod;
    return res;
}
```

使用条件与多测 代码要求 mod 为质数且主元非零才可用 Fermat 逆元；合数模必须换无除法算法或辗转消元，不能照抄这份代码。每组完整赋矩阵，det 会原地修改它；空矩阵行列式为 1。ll 乘积若可能溢出应转 128 位。时间 $O(n^3 + n \log mod)$。

5.3.3 矩阵树定理

General Form: The sum of weights of all spanning trees is given by the cofactor of the Laplacian matrix:

$$\sum_{T \in \mathcal{T}} \prod_{e \in T} w_e = \det(L^*)$$

1. Undirected Graph

$$L_{ij} = \begin{cases} \deg(i) & i = j \\ -w_{ij} & i \neq j \end{cases}$$

Result: $\det(L)$ after removing **any** k-th row and column.

2. Directed Graph (Root r) Remove the r-th row and column from L to calculate determinant.

A. Out-Tree ($r \to$ all): Use **In-Degree** & A^T.

$$L_{ij} = \begin{cases} \deg^{in}(i) & i = j \\ -w_{ji} & i \neq j \quad (\text{edge } j \to i) \end{cases}$$

B. In-Tree (all $\to r$): Use **Out-Degree** & A.

$$L_{ij} = \begin{cases} \deg^{out}(i) & i = j \\ -w_{ij} & i \neq j \quad (\text{edge } i \to j) \end{cases}$$

当然，此处的 w 可以替换为任意有交换律的东西，比如形式幂级数。

```
void solve(){
    n=d,m=d;ll o=d;
    f(i,1,n)f(j,1,n)a[i][j]=0;//0 无向；1 有向且 1 为根外向树
    if(o==0){
        f(i,1,m){
            ll u=d,v=d,w=d;
            a[u][u]+=w,a[v][v]+=w;
            a[u][v]-=w,a[v][u]-=w;
        }
        f(i,2,n)f(j,2,n)a[i-1][j-1]=a[i][j];
        printf("%lld\n",det(n-1,a));
    }if(o==1){
        f(i,1,m){
            ll u=d,v=d,w=d;
            a[v][v]+=w;
            a[v][u]-=w;
        }
        f(i,2,n)f(j,2,n)a[i-1][j-1]=a[i][j];
        printf("%lld\n",det(n-1,a));
    }
}
```

使用条件与多测 每组必须先清整个当前 $n \times n$ 矩阵，已在 solve 开头补上；自环对生成树无贡献，建矩阵时应忽略（代码的加减会抵消）。平行边权相加；加权度数是边权和而非边数。原矩阵压缩与 det 均破坏输入；多次删不同根时须从副本构造。权值要处于交换环中，仅“有交换律”不够；若调用前文 Fermat 消元，仍要求素数模，形式幂级数不能直接套除法实现。$n = 1$ 答案为 1。

5.3.4 BEST 定理 (BEST Theorem)

BEST 定理，用于求解**有向欧拉图**的欧拉回路计数。设 $G = (V, E)$ 是有向欧拉图（入度等于出度且连通），t_{root} 为以任意点 $root$ 为根的**根向树形图**（Arborescence）个数，则 G 的欧拉回路总数 $ec(G)$ 为：

$$ec(G) = t_{root} \cdot \prod_{v \in V} (\deg(v) - 1)!$$

注：t_{root} 通常配合**矩阵树定理**（Matrix Tree Theorem）求解。

使用条件与多测 计数约定必须明确：此式计数边可区分、固定首边的欧拉回路；若只固定起点而首边任选，再乘根的出度；若边不可区分不能随意套式。忽略零度孤立点后检查有边部分强连通且每点入度等于出度，deg 为出度。阶乘只对有边顶点计算；空图按题意单独定义。每组重建度数、矩阵树计数，固定模数的阶乘表可复用。

5.3.5 LGV 引理 (Weighted LGV Lemma)

LGV 引理，适用于 **DAG** 上带权不相交路径问题。设边权为 $w(e)$（可为数值或多项式），定义路径 P 的权值为 $\omega(P)=\prod_{e\in P} w(e)$。构造矩阵 M，其中 M_{ij} 为起点 a_i 到终点 b_j 的所有路径权值之和：

$$M_{ij}=\sum_{P:a_i\to b_j}\omega(P)=\sum_{P:a_i\to b_j}\prod_{e\in P}w(e)$$

若图满足条件（如平面图且起点终点排序使得仅 $\sigma=id$ 存在不相交路径），则所有不相交路径组 $(P_1,\dots,P_n)$ 的权值积之和为：

$$\det(M)=\sum_{(P_1,\dots,P_n)\in\text{Disjoint}}\prod_{k=1}^{n}\omega(P_k)$$

注：若求不相交路径总数，设 $w(e)=1$；若求路径权值总和，设 $w(e)=x^{val}$ 并做多项式运算。

使用条件与多测 一般 LGV 行列式给出带排列符号的点不相交路径族和；只有证明非恒等配对均不可能时才等于无符号计数，平面性本身不充分。这里不相交指顶点不相交。每组重建 DAG 路径 DP 和 M；对各源沿拓扑序传播，别把每条路径显式枚举。环图不能直接按本式做有限路径和。

5.4 多项式 (Polynomials)

统一使用 vector 版本：系数下标为次数，所有精度参数 n 指保留 n 项（模 x^n），不再混用最高次数。固定模数 998244353，系数输入须归一化到 $[0,mod)$。每次运算的临时数组为局部变量，无需多测清空；卷积为 $O(n\log n)$，Newton 求逆 / 对数 / 指数 / 幂及除法为相应精度的 $O(n\log n)$。NTT 长度不超过 2^{23}；Ln 要求常数项为 1，Exp 要求常数项为 0，Inv 和除法的分母须可逆。

```
namespace poly {
const ll mod=998244353;
using Poly=vector<ll>;
ll qp(ll a,ll b=mod-2){ll r=1;for(;b;b>>=1,a=a*a%mod)if(b&1)r=r*a%mod;return r;}
void NTT(Poly& a,bool inverse){
    int n=a.size();assert(n&&!(n&(n-1))&&n<=(1<<23));
    for(int i=1,j=0;i<n;i++){
        int bit=n>>1;for(;j&bit;bit>>=1)j^=bit;j^=bit;
        if(i<j)swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        ll root=qp(3,(mod-1)/len);if(inverse)root=qp(root);
        for(int i=0;i<n;i+=len){
            ll w=1;
            for(int j=0;j<len/2;j++,w=w*root%mod){
                ll u=a[i+j],v=a[i+j+len/2]*w%mod;
                a[i+j]=(u+v)%mod;a[i+j+len/2]=(u-v+mod)%mod;
            }
        }
    }
    if(inverse){ll inv=qp(n);for(ll &x:a)x=x*inv%mod;}
}
Poly Mul(Poly a,Poly b){
    if(a.empty()||b.empty())return {};
    int size=a.size()+b.size()-1;
    if(min(a.size(),b.size())<=32){
```

```
        Poly c(size);for(int i=0;i<int(a.size());i++)for(int j=0;j<int(b.size());j++)c[i+j]=(c[i+j]+
    a[i]*b[j])%mod;
        return c;
    }
    if(size>(1<<23))throw length_error("NTT capacity exceeded");
    int n=1;while(n<size)n<<=1;a.resize(n);b.resize(n);
    NTT(a,false);NTT(b,false);
    for(int i=0;i<n;i++)a[i]=a[i]*b[i]%mod;
    NTT(a,true);a.resize(size);return a;
}
Poly Inv(const Poly& a,int n=-1){
    if(n<0)n=a.size();if(!n)return {};
    assert(!a.empty()&&a[0]);Poly g{qp(a[0])};
    while(int(g.size())<n){
        int k=min(n,int(g.size())*2);Poly prefix(a.begin(),a.begin()+min(k,int(a.size())));
        Poly t=Mul(prefix,g);t.resize(k);
        for(ll &x:t)x=(mod-x)%mod;t[0]=(t[0]+2)%mod;
        g=Mul(g,t);g.resize(k);
    }
    return g;
}
Poly Dev(const Poly& a){
    if(a.size()<2)return {};
    Poly b(a.size()-1);for(int i=1;i<int(a.size());i++)b[i-1]=a[i]*i%mod;return b;
}
Poly Int(const Poly& a){
    assert(a.size()<size_t(mod));Poly b(a.size()+1),iv(a.size()+1);
    if(!a.empty())iv[1]=1;
    for(int i=2;i<=int(a.size());i++)iv[i]=(mod-mod/i)*iv[mod%i]%mod;
    for(int i=0;i<int(a.size());i++)b[i+1]=a[i]*iv[i+1]%mod;return b;
}
Poly Ln(const Poly& a,int n){
    if(!n)return {};assert(!a.empty()&&a[0]==1);
    Poly b=Mul(Dev(a),Inv(a,n));b.resize(n-1);b=Int(b);b.resize(n);return b;
}
Poly Exp(const Poly& a,int n){
    if(!n)return {};assert(a.empty()||a[0]==0);Poly g{1};
    while(int(g.size())<n){
        int k=min(n,int(g.size())*2);Poly log=Ln(g,k),t(k);
        for(int i=0;i<k;i++)t[i]=((i<int(a.size())?a[i]:0)-log[i]+mod)%mod;
        t[0]=(t[0]+1)%mod;g=Mul(g,t);g.resize(k);
    }
    return g;
}
Poly Pow(const Poly& a,unsigned long long exponent,int n){
    if(!n)return {};Poly result(n);
    if(!exponent){result[0]=1;return result;}
    int lead=0;while(lead<int(a.size())&&!a[lead])lead++;
    if(lead==int(a.size())||(__uint128_t)lead*exponent>=unsigned(n))return result;
    int shift=int((__uint128_t)lead*exponent),k=n-shift;
    ll c=a[lead],inv=qp(c);Poly unit(a.begin()+lead,a.end());unit.resize(k);
    for(ll &x:unit)x=x*inv%mod;
    Poly log=Ln(unit,k);for(ll &x:log)x=x*(exponent%mod)%mod;
    Poly b=Exp(log,k);ll scale=qp(c,exponent%(mod-1));
    for(int i=0;i<k;i++)result[i+shift]=b[i]*scale%mod;return result;
}
void trim(Poly& a){while(!a.empty()&&!a.back())a.pop_back();}
pair<Poly,Poly> Div(Poly a,Poly b){
    trim(a);trim(b);if(b.empty())throw invalid_argument("zero divisor");
```

```
    if(a.size()<b.size())return {{},a};
    int k=a.size()-b.size()+1;Poly ar=a,br=b;
    reverse(ar.begin(),ar.end());reverse(br.begin(),br.end());ar.resize(k);
    Poly q=Mul(ar,Inv(br,k));q.resize(k);reverse(q.begin(),q.end());
    Poly product=Mul(q,b);for(int i=0;i<int(product.size());i++)a[i]=(a[i]-product[i]+mod)%mod;
    a.resize(b.size()-1);trim(a);trim(q);return {q,a};
}
} // namespace poly
```

使用条件与多测 接口相较旧指针版已变更：Mul(a,b)、Inv(a,n)、Ln(a,n)、Exp(a,n)、Pow(a,k,n)、Div(a,b)（返回商与余数）；输入不会被修改。零多项式在 trim/Div 中可表示为空 vector。0^0 约定为 1；空精度 n=0 返回空。不同操作的中间卷积长度也须在 NTT 容量内，不能只按最终 n 判断。

5.4.1 Bostan-Mori

求 $[x^m]P(x)/Q(x)$；依赖前文 poly 命名空间，要求 $Q(0)\neq 0$ 且系数已归一化。按值传参保留原输入，局部状态支持直接多测，返回结果会乘 $Q(0)^{-1}$。零分子返回 0，避免筛奇数项后空分子越界。

```
ll bostan_mori(poly::Poly P,poly::Poly Q,unsigned long long m){
    using namespace poly;
    trim(P);trim(Q);assert(!Q.empty()&&Q[0]);
    if(P.empty())return 0;
    while(m){
        Poly neg=Q;for(int i=1;i<int(neg.size());i+=2)neg[i]=(mod-neg[i])%mod;
        Poly a=Mul(P,neg),b=Mul(Q,neg);P.clear();Q.clear();
        for(int i=int(m&1);i<int(a.size());i+=2)P.push_back(a[i]);
        for(int i=0;i<int(b.size());i+=2)Q.push_back(b[i]);
        trim(P);trim(Q);if(P.empty())return 0;m>>=1;
    }
    return P[0]*qp(Q[0])%poly::mod;
}
```

使用条件与多测 次数为 d 时通常为 $O(d\log d\log(m+1))$；输入分子次数很大时复杂度也依赖其长度。常数分母与 m=0 同样可用。

5.5 数值方法 (Numerical Methods)

5.5.1 自适应辛普森 (Adaptive Simpson's Rule)

自适应 Simpson，传入函数与容差；复用端点 / 中点值以减少函数求值。每次调用使用局部状态，多测须重新捕获本组函数参数。区间内须连续且足够光滑，不可含极点或未经处理的广义积分；给出绝对误差估计，不是对任意函数的严格误差保证。

```
template<class F>
double integrate(F func,double left,double right,double eps=1e-8,int max_depth=30){
    assert(eps>0&&max_depth>0);
    if(left==right)return 0;
    if(left>right)return -integrate(func,right,left,eps,max_depth);
    auto eval=[&](double x){double y=func(x);if(!isfinite(y))throw domain_error("non-finite
     integrand");return y;};
    auto simpson=[](double l,double r,double fl,double fm,double fr){return (r-l)*(fl+4*fm+fr)/6;};
    function<double(double,double,double,double,double,double,double,int)> rec;
```

```
    rec=[&](double l,double r,double fl,double fm,double fr,double old,double tol,int depth)->double
    {
        double m=l+(r-l)/2;
        if(m==l||m==r)return old;
        double lm=eval(l+(m-l)/2),rm=eval(m+(r-m)/2);
        double a=simpson(l,m,fl,lm,fm),b=simpson(m,r,fm,rm,fr),delta=a+b-old;
        if(fabs(delta)<=15*tol)return a+b+delta/15;
        if(!depth)throw runtime_error("Simpson did not converge");
        return rec(l,m,fl,lm,fm,a,tol/2,depth-1)+rec(m,r,fm,rm,fr,b,tol/2,depth-1);
    };
    double m=left+(right-left)/2,fl=eval(left),fm=eval(m),fr=eval(right);
    return rec(left,right,fl,fm,fr,simpson(left,right,fl,fm,fr),eps,max_depth);
}
```

6 Misc

6.1 C++ 手动上下取整

利用 C++ 向零截断的商和余数修正，避免原版 $(x+y-1)$ 以及对 LLONG_MIN 取负导致溢出。除零和 LLONG_MIN/-1 的不可表示商抛异常，无可变状态，可直接多测。

```
ll Floor(ll x,ll y){
    if(!y||(x==LLONG_MIN&&y==-1))throw domain_error("invalid division");
    ll q=x/y,r=x%y;
    return q-(r!=0&&((r<0)!=(y<0)));
}
ll Ceil(ll x,ll y){
    if(!y||(x==LLONG_MIN&&y==-1))throw domain_error("invalid division");
    ll q=x/y,r=x%y;
    return q+(r!=0&&((r<0)==(y<0)));
}
```

6.2 单点修改区间摩尔投票

线段树维护摩尔投票过程，节点 (c, num) 表示投票完以后剩下的数和个数（没有的话记为 -1）。合并时若候选相同则计数相加；若候选不同，则抵消较小计数，保留计数较大的候选以及两个计数之差；相等时计数为 0（无候选）。只选较大的计数而不减去较小计数是错误的。

然后还有一步要检验出来的数是不是合法，需要维护一个区间出现某个数的次数。可以平衡树。

```
struct Vote{ll candidate;int count;};
Vote merge_vote(Vote a,Vote b){
    if(!a.count)return b;if(!b.count)return a;
    if(a.candidate==b.candidate)return {a.candidate,a.count+b.count};
    if(a.count>=b.count)return {a.candidate,a.count-b.count};
    return {b.candidate,b.count-a.count};
}
```

使用条件与多测 仅保证出现次数严格超过区间长度一半的元素会成为候选，必须额外查真实出现次数后验证。每组重建线段树及“值到有序位置集合”的索引，修改时同步删旧位置、加新位置；不存在候选宜用 num=0 表示，不要假设 -1 永远不是合法值。

6.3 双栈模拟 dequeue

两个栈，左边的一个维护当前位置到栈底的前缀操作和，右边一个维护栈底到当前位置的前缀操作和。查询就是两个栈顶的维护值操作一下，首尾 add 都是简单的。

困难的在于某个栈空了的时候的删除操作。把另一个栈的元素切成等大小的左右两段，暴力重构即可均摊 $O(1)$。

使用条件与多测 聚合操作要求结合律；若不交换，左右栈必须维护相反的累积方向并按序合并，不能交换拼接顺序。每组清两个栈及聚合缓存；空队列删除 / 查询须单独定义。重构时保持原序列顺序，只在一侧为空时均分，端点操作摊还 $O(1)$。

6.4 测速

```cpp
auto start = std::chrono::steady_clock::now();

// main code here

auto end = std::chrono::steady_clock::now();
std::chrono::duration<double> elapsed = end - start;
std::cerr << "\nRunning: " << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).
    count() << " ms" << std::endl;
```

使用条件与多测 使用单调 steady_clock，避免系统时间调整影响耗时。测单组时将 start/end 放入 solve 内并逐组重建；测整体时放在多测循环外。计时本身不涉及算法状态清理，输出到 cerr 不计入答案输出。

6.5 KKT

$$\begin{aligned} \min_{x\in\mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1,\dots,m \\ & h_j(x) = 0, \quad j = 1,\dots,p \end{aligned}$$

KKT:

1. Stationarity: $\nabla f(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^{p} \nu_j^* \nabla h_j(x^*) = 0$
2. Primal Feasibility: $g_i(x^*) \le 0, \quad h_j(x^*) = 0$
3. Dual Feasibility: $\lambda_i^* \ge 0$
4. Complementary Slackness: $\lambda_i^* g_i(x^*) = 0$

使用条件与多测 KKT 成为局部最优的必要条件需要约束规范性等条件；一般非凸问题即便满足 KKT 也不保证全局最优。可微凸目标、凸不等式约束、仿射等式约束时，满足 KKT 是全局最优的充分条件，Slater 等条件可保证合适的必要性。每次求解须重建约束及乘子初值；此条为数学条件，无通用求解器或清空代码。